

Building the NH-7 Demo Model

Complete guide, from the Mey inventory to a live **/risk-map** — with every script
NH-7 (Rishikesh–Joshimath) | static terrain model + live-rainfall layer | October 2026

From inventory to live risk map



tested end to end on mocked services using your real 309 coordinates; live API calls are verified on your first run

How to use this PDF

Run the scripts as **.py files** (delivered alongside this PDF). Do not copy code out of the PDF: PDF text loses indentation. The appendices contain the exact same code for reading and review.

1. Where you stand

An honest audit of every dataset you have, and what each one is good for:

What you have	Verdict	Use it for
Mey et al. (2024) inventory , 309 road-blocking landslides on NH-7	The best data you have: surveyed, accurately located, peer-reviewed, covers the whole road.	Positives for the terrain model.
NASA COOLR events, 38 with dates	Real dates, but news-geocoded: only 7 of 38 are located within 1 km; 17 are 10 km, 25 km, 50 km or unknown.	/history map markers only, with an accuracy caveat.
NGDR points, 28	Unclear whether the layer is an event inventory or a terrain classification; small.	Not needed. Leave out.
CHIRPS 'dynamic' dataset, 76 rows	Date-matched negatives remove weather by design (pair correlation 0.83), so it cannot teach rainfall -> risk. Negatives hug the road while positives scatter: distance alone scored AUC 0.66–0.68.	Do not train on it.

Is it enough for a demo?

Yes — for a model that ranks road segments by terrain susceptibility, with a live-rainfall layer on top. **No** — for a claim that a model has learned how rainfall triggers landslides on NH-7. The scripts and the pitch are built around that difference, and that difference is what keeps you safe from a hard question.

2. What changes in your current data

- Positives = the Mey 309**, not the NASA/NGDR mix. Accurate locations matter because terrain is computed at the point.

- 2 **Negatives are sampled on the real road** (OpenStreetMap), each pushed sideways by a distance copied from the positives, and re-checked by *measured* distance. A **leak test** then proves that distance to the road alone cannot predict the label.
 - 3 **Terrain from the 30 m Copernicus DEM** (the DEM Mey et al. used for their slope covariate) instead of the 90 m Open-Meteo API, plus distance to rivers/streams. Step 1 still works without it, so you always have a fallback.
 - 4 **No rainfall inside the model.** Rainfall is applied at serving time as a labelled, adjustable rule (section 6).
 - 5 **Spatial-block cross-validation** with a 2 km buffer replaces random splits: whole stretches of road are held out, as recommended by Kumar et al. (2025) for regional landslide models.
 - 6 **Aspect bug fixed.** Earlier scripts used `atan2(-dz_dx, dz_dy)`, which swaps north and south. The new code uses the downslope bearing and is unit-tested on four planes. Aspect is now stored as sin/cos.
-

3. More data: what is actually worth getting

Ranked by expected benefit. More landslide *points* is not at the top: 309 positives is plenty for a model with around ten features. Feature quality and negative sampling are the bottleneck.

1. The Mey et al. supplement (free, biggest win)

The paper reports that its model uses **lithozones (34 lithologies), major faults, road widening and rainfall** as predictors, and states that the code and data to run the analysis are in the Supplement (about 140 MB for the preprint). Lithology and road widening are two of the four main controls they report, so this is the most promising upgrade.

```
# 1. Download the Supplement from https://nhess.copernicus.org/articles/24/3207/2024/ and unzip it
# 2. See what is inside each layer (columns, geometry, CRS):
python add_features.py --inspect lithozones.shp faults.shp widened_roads.shp
# 3. Use them (file names and column names will differ: take them from step 2):
python add_features.py --litho lithozones.shp --litho-field LITHO --faults faults.shp \
    --widened widened_roads.shp --widened-query "widened == 1"
```

Honest status of this part

The supplement's real files were not available when this guide was written, and geopandas could not be run in the test environment, so this code path is **completely untested**. It needs `pip install geopandas`. Run `--inspect` first and check the column names. If the supplement also contains the road line, convert it to GeoJSON (EPSG:4326) and save it as `nh7_road_osm.geojson`: step 1 will use it instead of OpenStreetMap.

2. 30 m Copernicus DEM (scripted, free)

Downloaded automatically from the public AWS bucket `copernicus-dem-30m` (two 1° tiles cover the corridor), then mosaicked so points near a tile edge still get a full neighbourhood. Gives Horn slope, aspect, curvature, local relief, TPI (ridge/valley position) and the **steepest slope within 210 m**, in the spirit of the slope covariate in Mey et al., who take the hillslope gradient within 210 m of the road (check the paper for the exact statistic).

3. Rivers and streams (scripted, free)

Distance to the nearest OpenStreetMap river/stream: undercutting of slopes at the toe is a classic landslide control. OSM coverage can be incomplete, so the script compares feature sets and keeps this only if it helps.

4. More landslide points, if you still want them

- GSI Bhukosh / NRSC Bhuvan inventories: access has been unreliable for you. If you get them, remove anything within ~200 m of a Mey point (same events) and keep a source column so you can test with and without.

- Your own digitising from Google Earth: fine, but label it as such and keep it out of the validation blocks.
- **Do not add** the NASA points to training, and never pad the set with synthetic points: it inflates scores without adding information.

5. Later, not for the demo

Land cover (ESA WorldCover), soil and NDVI need very large rasters and add risk for little gain in 8 days. A genuinely *dynamic* model needs accurately located events with exact dates across many storms; none of your files have that. That is the research-paper direction, not a demo task.

4. Setup

```
nh7_model/
build_static_dataset.py  add_features.py  nh7_features.py  score_segments.py  risk_service.py
nh7_published_309_inventory.csv    <- Mey et al. positives (columns: latitude, longitude, ...)
nh7_segments.csv              <- route polyline ordered Rishikesh -> Joshimath (longitude, latitude)
```

```
pip install numpy pandas requests scipy scikit-learn matplotlib joblib
pip install rasterio          # step 2 (30 m DEM)
pip install geopandas         # only if you use the Mey layers
```

Check your `nh7_segments.csv`: rows must run from Rishikesh to Joshimath in order. The script warns if they do not look ordered. A straight-line polyline is fine: it is only the reference for 'position along the road' and for the leak test.

5. Run order, and how to read each result

Step 0: smoke test (2 minutes)

```
python build_static_dataset.py --limit 30
```

Uses 30 positives only. It should print which road mode it found, fetch terrain, and show a CV table. Any red flag here is cheaper to fix now than after the full run.

Step 1: build the dataset (about 5–10 minutes)

```
python build_static_dataset.py
```

You see	It means	Do this
MODE A	Negatives lie on the real road (OSM matched your positives).	Good. Continue.
MODE B	OSM was unreachable or did not match; negatives are offset-matched around your polyline, not guaranteed to be on the road.	Works for a demo. To get Mode A: Overpass Turbo export, saved as <code>nh7_road_osm.geojson</code> (the script prints the query).
LEAK TEST: PASS	Distance to the road alone gets AUC within 0.07 of 0.5.	Continue.
LEAK TEST: WARN	Distance still predicts the label, so terrain results cannot be trusted.	Stop. Check Mode, the offset matching, and that positives and negatives use the same route file.
elevation check: CHECK	API elevation disagrees strongly with the inventory's own elevation.	Check lat/lon order and the elevation API.
pooled AUC < 0.58	No demonstrated skill from terrain alone at this resolution.	Do not oversell it. Run step 2 and try the Mey layers.

You see	It means	Do this
pooled AUC 0.58–0.70	Weak but real ranking ability.	Usable as a relative ranking if you say 'weak'.
pooled AUC >= 0.70	Moderate ranking ability on held-out stretches.	Good for a demo; still say 'terrain ranking'.

Pooled vs block AUC. Pooled ranks points across the whole corridor; block AUC ranks points within each stretch. Elevation can look strong per block but weak pooled because it rises steadily towards Joshimath. Chance levels are printed under the table (AUC 0.5, top-20% capture 20%).

Step 2: better terrain and extra layers (about 10 minutes plus the DEM download)

```
python add_features.py --check-dem      # smoke test: expect Rishikesh 300-500 m, Joshimath 1700-2100 m
python add_features.py
```

It prints the median elevation error against the inventory for both DEMs, then a CV table for feature sets A (90 m), B (30 m), C (+ rivers) and D (+ Mey layers, if given). It keeps the **simplest** set that beats the previous one by more than 0.01 pooled AUC, so noise cannot talk you into extra features.

Step 3: score the road (about 2 minutes)

```
python score_segments.py
```

Prints the 18 segments with a terrain percentile and a plain-language driver (for example *high slope*, $z=+1.8$). A segment with no road points is scored along its straight chord and marked chord, because 'no data' must never look like 'safe'. The face-validity lines check where Sirobagarh ranks: it is a documented chronic slide zone, so a very low rank deserves a second look. It is a sanity check, not a validation.

Why percentiles, and why that is a limit

The model's output is a ranking score, not a probability. The percentile therefore always spreads 0–1 across the 18 segments. The map shows **relative** terrain susceptibility on this corridor, never an absolute chance of a slide.

6. Wire it into the backend

- 1 Copy `risk_service.py` and `segment_static_scores.json` into your FastAPI app/ folder.
- 2 In the `/risk-map` route, call `get_live_risk_map()` and keep your existing mock as the fallback (the file header has the exact snippet).
- 3 Test: `curl localhost:8000/risk-map`, then `curl "localhost:8000/risk-map?simulate_rain_mm=120"`.

Part	Rule	Why
Terrain	terrain_percentile from the model (0 = safest segment, 1 = riskiest)	Ranking, not probability
Rain	rain_index = min(rain_mm / 100, 1); rain_mm = yesterday + today + tomorrow at the segment midpoint (Open-Meteo)	Simple, visible, adjustable
Score	$0.6 \times \text{terrain} + 0.4 \times \text{rain_index}$	Dry days cap at High; Very High needs terrain AND rain
Levels	≥ 0.75 Very High, ≥ 0.50 High, ≥ 0.25 Moderate, else Low	Same four labels as the API contract
Rain service down	Terrain-only score; rain_status = unavailable	Degrades honestly
Demo switch	?simulate_rain_mm=120 labels every segment simulated	Shows the storm effect on a dry day without faking data

Say this out loud

The weights (0.6/0.4) and the 100 mm reference are **demo heuristics**: not learned, not calibrated. If you want a defensible number, replace them with a published Himalayan rainfall threshold, or with the rainfall coefficient from the Mey et al. supplement, and cite it.

The response keeps your API contract (id, name, sequence_order, start_coords, end_coords, risk_level, risk_score, updated_at) and adds fields your friends can show: terrain_percentile, rain_mm_3d, rain_status, main_driver, method. Extra keys are safe for existing clients to ignore, and main_driver is exactly the 'why is this risky?' feature from your team brief.

7. What you can and cannot claim

You can say	Do not say
Built on the peer-reviewed Mey et al. (2024) inventory of 309 road-blocking landslides, about 1.25 per km.	'Our AI predicts landslides' or 'early warning' (it ranks terrain; it does not forecast events).
Validated with spatial-block cross-validation: whole stretches held out, not random points.	Any accuracy % from a random split.
Negatives matched on distance to the road, and leak-tested.	'The model learned how rainfall triggers slides.'
Terrain ranking from the 30 m Copernicus DEM (plus rivers).	AUC numbers from mocked test runs (they mean nothing).
The rainfall layer is an explicit, adjustable rule, shown in every response.	'Validated by experts / agencies.'

Likely judge questions, with honest answers

Question	Answer
Is the rainfall part machine learning?	No. Terrain is a trained model; rainfall is a transparent rule on top. We say so because the available data cannot teach rainfall response.
How accurate is it?	Report the pooled block AUC from your own run, name it a ranking score, and say what chance is (0.5).
Why not satellite images / deep learning?	That is our research track. For the demo, a small tabular model is honest, fast and explainable.
Where does the data come from?	Mey et al. 2024 (landslides), Copernicus DEM (terrain), OpenStreetMap (road, rivers), Open-Meteo (rainfall).
What would you do with more time?	Add lithology and road widening from the Mey layers, calibrate rainfall from dated events, validate with field reports.

8. Troubleshooting

Symptom	Likely cause	Fix
Overpass fails or returns nothing	Server busy, or the road is tagged differently (older data may say NH 58).	Retry later; use the Overpass Turbo export; or accept Mode B.
Open-Meteo 429 / 400	Rate limit, or too many coordinates in a request.	The script retries and halves the batch. Also try --batch-pts 5.
LEAK TEST WARN	Negatives not matched to the positives' distance.	See section 5; do not continue until it passes.

Symptom	Likely cause	Fix
elevation check CHECK	Swapped lat/lon, or API problem.	Print the first rows; compare with Google Earth.
rasterio will not install (Windows)	Missing wheel for your Python version.	Use a supported Python (3.9–3.12) or conda. Or skip step 2: step 1's model works.
Fewer rows after step 2	Points within ~150 m of the DEM edge have no full neighbourhood.	Normal for a few points; large losses mean a tile problem.
AUC about 0.5 or lower	Terrain alone does not separate the classes at this resolution.	Add the Mey layers; do not fake a better number.
A segment says 'chord'	No road points fall in that segment.	Check the road cache covers Joshimath; treat that score with caution.

9. What was tested, and what was not

Checked (mocked services, your real 309 coordinates)	Not verified live (you will see it on your first run)
Negative sampling and the leak test: a deliberately biased design is flagged, a matched one passes.	Overpass queries for the road and for rivers (including the NH 7 / NH 58 tag pattern).
Block CV, model and feature-config saving, Mode A / Mode B / bad-road fallback.	Open-Meteo elevation with 100 coordinates per request (assumed; the script halves the batch automatically).
Slope/aspect/curvature/TPI against analytic planes, a bowl and a peak; edge handling.	The Copernicus tile download and rasterio mosaic (use --check-dem).
Stream distances, download glue, retry on 429, batch halving, missing-rasterio message.	geopandas joins for the Mey layers (schema unknown: use --inspect).
Segment scoring, chord fallback, risk formula, outage degradation, cache, simulate mode.	The Open-Meteo forecast response for 18 locations; and the exact route code in your Antigravity backend.

Read this before quoting any number

The AUC values printed while I tested come from **synthetic terrain** and mean nothing. Your real numbers will be different, and may be low. That is a result, not a failure: report it plainly.

Relation to the paper. This is the hackathon model. The image-segmentation model for Uttarakhand and Himachal Pradesh remains a separate research track, and the Mey inventory is points, not the polygon masks segmentation needs.

References and data licences

- Mey, J., Guntu, R. K., Plakias, A., Silva de Almeida, I., and Schwanghart, W. (2024). More than one landslide per road kilometer – surveying and modelling mass movements along the Rishikesh–Joshimath (NH-7) highway, Uttarakhand, India. *Natural Hazards and Earth System Sciences*, 24, 3207. <https://doi.org/10.5194/nhess-24-3207-2024> (cite it, and check the supplement's licence before redistributing the data).
- Kumar, C., Walton, G., Santi, P., and Luza, C. (2025). Random cross-validation produces biased assessment of machine learning performance in regional landslide susceptibility prediction. *Remote Sensing*, 17(2), 213. <https://doi.org/10.3390/rs17020213>
- Copernicus DEM GLO-30, European Space Agency / Copernicus programme; public copy on AWS Open Data (bucket copernicus-dem-30m). Attribution is required: check the licence text before publishing.
- OpenStreetMap contributors (road and waterway geometry), Open Database Licence (ODbL): attribution required.
- Open-Meteo (elevation and forecast APIs), <https://open-meteo.com>: attribution required; free tier is for non-commercial use.
- Horn, B. K. P. (1981). Hill shading and the reflectance map. *Proceedings of the IEEE*, 69(1), 14–47 (slope and aspect method).

Appendix A: build_static_dataset.py (825 lines)

Step 1: builds the dataset (Mey positives, road-matched negatives, 90 m terrain), runs the leak test and spatial-block CV, saves the baseline model.

```
#!/usr/bin/env python3
"""
NH-7 STATIC landslide-susceptibility dataset + spatial-block CV
=====

Positives : Mey et al. (2024) inventory of 309 road-blocking landslides along
             NH-7 (Rishikesh-Joshimath), file: nh7_published_309_inventory.csv
Negatives : background points on the real road (OpenStreetMap) whose distance
             to the road mirrors the positives' distance distribution.
             If OSM is unreachable, falls back to offset-matched sampling around
             your nh7_segments.csv polyline ("Mode B").
Features   : terrain only (elevation, slope, aspect, local relief) from the
             Open-Meteo 90 m elevation API. NO rainfall: rainfall is applied at
             serving time in the backend, not learned here.
Checks     : (1) leak test - distance-to-route alone must NOT predict the label,
             (2) terrain source vs. the inventory's own 30 m DEM elevation,
             (3) spatial-block CV along the road with a buffer between blocks.

Quick test run: python build_static_dataset.py --limit 30
Full run:       python build_static_dataset.py

Citation (keep in your pitch / paper):
Mey, J., Guntu, R. K., Plakias, A., Silva de Almeida, I., and Schwanghart, W.
(2024). More than one landslide per road kilometer - surveying and modelling
mass movements along the Rishikesh-Joshimath (NH-7) highway, Uttarakhand,
India. Nat. Hazards Earth Syst. Sci., 24, 3207. doi:10.5194/nhess-24-3207-2024
(Check the supplement's license/terms before redistributing the data.)
"""

from __future__ import annotations

import argparse
import json
import math
import random
import sys
import time
from datetime import datetime, timezone
from pathlib import Path

import joblib
import matplotlib

matplotlib.use("Agg")
import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
import requests
from scipy.spatial import cKDTree
from sklearn.ensemble import RandomForestClassifier
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import average_precision_score, roc_auc_score
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler

# -----
# CONFIG
# -----
INVENTORY_FILE = "nh7_published_309_inventory.csv"
SEGMENTS_FILE = "nh7_segments.csv"
ROAD_CACHE_FILE = "nh7_road_osm_cache.json"
ROAD_GEOJSON_FILE = "nh7_road_osm_geojson" # optional manual Overpass-Turbo export
TERRAIN_CACHE_FILE = "terrain_cache_static.json"

OUT_DATA = "nh7_static_training_data.csv"
OUT_META = "pipeline_meta_static.json"
OUT_NOTES = "DATA_NOTES_static.md"
OUT_PLOTS = "diagnostic_plots_static.png"
OUT_MODEL = "nh7_static_model.joblib"
OUT_CV = "cv_report_static.json"

SLOPE_OFFSET_M = 150.0 # finite-difference offset; DEM is 90 m so stay > 90
FEATURES = ["elevation_m", "slope_deg", "aspect_sin", "aspect_cos", "local_relief_m"]

OVERPASS_URLS = [
    "https://overpass-api.de/api/interpreter",
    "https://overpass.kumi.systems/api/interpreter",
]
OPEN_METEO_ELEV = "https://api.open-meteo.com/v1/elevation"

LEAK_MAX_DEVIATION = 0.07 # |AUC - 0.5| of distance-to-route alone must stay below this
OSM_ACCEPT_SHARE = 0.80 # share of positives within 300 m of OSM road
OSM_ACCEPT_DIST_KM = 0.30
MAX_OFFSET_KM = 10.0 # sanity cap on lateral offsets copied from positives

CITATION = (
    "Mey, J., Guntu, R. K., Plakias, A., Silva de Almeida, I., and Schwanghart, W. (2024). "
```

```

"the Rishikesh-Joshimath (NH-7) highway, Uttarakhand, India. Nat. Hazards Earth Syst. Sci., "
"24, 3207. https://doi.org/10.5194/nhess-24-3207-2024"
)

# -----
# GEOMETRY
# -----
class Proj:
    """Local equirectangular projection to kilometres (fine at corridor scale)."""

    def __init__(self, lat0: float):
        self.kx = 111.320 * math.cos(math.radians(lat0))
        self.ky = 110.574

    def xy(self, lat, lon) -> np.ndarray:
        return np.column_stack([np.asarray(lon, float) * self.kx, np.asarray(lat, float) * self.ky])

    def latlon(self, xy: np.ndarray):
        return xy[:, 1] / self.ky, xy[:, 0] / self.kx

def densify_polyline(xy: np.ndarray, step_km: float):
    """Points every step_km along a polyline, with a unit tangent for each point."""
    pts, tans = [], []
    for a, b in zip(xy[:-1], xy[1:]):
        d = b - a
        length = float(np.hypot(*d))
        if length == 0:
            continue
        t = d / length
        s = np.arange(0.0, length, step_km)
        pts.append(a + s[:, None] * t)
        tans.append(np.tile(t, (len(s), 1)))
    if not pts:
        return np.empty((0, 2)), np.empty((0, 2))
    return np.vstack(pts), np.vstack(tans)

def project_to_polyline(q_xy: np.ndarray, poly_xy: np.ndarray):
    """Distance (km), signed distance, along-line position (km) of each query point."""
    a, b = poly_xy[:-1], poly_xy[1:]
    d = b - a
    l2 = (d ** 2).sum(1)
    seg_len = np.sqrt(l2)
    cum = np.concatenate([[0.0], np.cumsum(seg_len)])
    best = np.full(len(q_xy), np.inf)
    along = np.zeros(len(q_xy))
    sign = np.ones(len(q_xy))
    for i in range(len(a)):
        if l2[i] == 0:
            continue
        w = q_xy - a[i]
        t = np.clip((w @ d[i]) / l2[i], 0.0, 1.0)
        proj = a[i] + t[:, None] * d[i]
        dist = np.hypot(q_xy[:, 0] - proj[:, 0], q_xy[:, 1] - proj[:, 1])
        cross = d[i, 0] * w[:, 1] - d[i, 1] * w[:, 0]
        better = dist < best
        best[better] = dist[better]
        along[better] = cum[i] + t[better] * seg_len[i]
        sign[better] = np.where(cross[better] >= 0, 1.0, -1.0)
    return best, best * sign, along, float(cum[-1])

# -----
# INPUTS
# -----
def load_inventory(path: str) -> pd.DataFrame:
    inv = pd.read_csv(path)
    low = {c.lower(): c for c in inv.columns}
    lat = next((low[k] for k in ("latitude", "lat") if k in low), None)
    lon = next((low[k] for k in ("longitude", "lon", "lng") if k in low), None)
    if lat is None or lon is None:
        raise ValueError(f"{path}: need latitude/longitude columns, found {list(inv.columns)}")
    inv = inv.rename(columns={lat: "latitude", lon: "longitude"})
    inv["latitude"] = pd.to_numeric(inv["latitude"], errors="coerce")
    inv["longitude"] = pd.to_numeric(inv["longitude"], errors="coerce")
    inv = inv.dropna(subset=["latitude", "longitude"]).drop_duplicates(["latitude", "longitude"])
    return inv.reset_index(drop=True)

def load_segments(path: str) -> pd.DataFrame:
    seg = pd.read_csv(path)
    low = {c.lower(): c for c in seg.columns}
    lat = next((low[k] for k in ("latitude", "lat") if k in low), None)
    lon = next((low[k] for k in ("longitude", "lon", "lng") if k in low), None)
    if lat is None or lon is None:
        raise ValueError(f"{path}: need latitude/longitude columns, found {list(seg.columns)}")
    seg = seg.rename(columns={lat: "latitude", lon: "longitude"}).dropna(subset=["latitude", "longitude"])
    seg = seg.reset_index(drop=True)
    if len(seg) < 2:
        raise ValueError(f"{path}: need at least 2 route vertices")
    trend = abs(np.corrcoef(np.arange(len(seg)), seg["longitude"])[0, 1])
    if trend < 0.9:
        print(f" [WARN] {path} rows do not look ordered from Rishikesh to Joshimath "
              f"({|corr(row, longitude)|={trend:.2f}). Along-route blocks may be wrong.")

```

```

return seg

# -----
# ROAD GEOMETRY (OpenStreetMap)
# -----
def overpass_query(bbox):
    s, w, n, e = bbox
    return (
        "[out:json][timeout:90];\n"
        f'way["highway"]["ref"~"NH ?(7|58)(;.*)?$"]({s:.4f},{w:.4f},{n:.4f},{e:.4f});\n'
        "out geom;"
    )

def parse_overpass_json(payload):
    ways = []
    for el in payload.get("elements", []):
        if el.get("type") != "way":
            continue
        if str((el.get("tags") or {}).get("highway", "")).endswith("_link"):
            continue
        geom = el.get("geometry") or []
        if len(geom) >= 2:
            ways.append([(g["lat"], g["lon"]) for g in geom])
    return ways

def parse_geojson(gj):
    ways = []
    for f in gj.get("features", []):
        if str((f.get("properties") or {}).get("highway", "")).endswith("_link"):
            continue
        g = f.get("geometry") or {}
        if g.get("type") == "LineString":
            lines = [g["coordinates"]]
        elif g.get("type") == "MultiLineString":
            lines = g["coordinates"]
        else:
            continue
        for line in lines:
            if len(line) >= 2:
                ways.append([(c[1], c[0]) for c in line])
    return ways

def load_road_ways(bbox, session, use_osm=True):
    """Returns (ways or None, description). Order: cache -> manual geojson -> Overpass."""
    if not use_osm:
        return None, "OSM disabled by --no-osm"
    cache = Path(ROAD_CACHE_FILE)
    if cache.exists():
        try:
            data = json.loads(cache.read_text(encoding="utf-8"))
            if data.get("ways"):
                return [(tuple(p) for p in w) for w in data["ways"]], f"cache {ROAD_CACHE_FILE}"
        except Exception as e: # noqa: BLE001
            print(f" [WARN] could not read {ROAD_CACHE_FILE}: {e}")
    gj_path = Path(ROAD_GEOJSON_FILE)
    if gj_path.exists():
        try:
            ways = parse_geojson(json.loads(gj_path.read_text(encoding="utf-8")))
            if ways:
                return ways, f"manual export {ROAD_GEOJSON_FILE}"
        except Exception as e: # noqa: BLE001
            print(f" [WARN] could not read {ROAD_GEOJSON_FILE}: {e}")

    q = overpass_query(bbox)
    last = "no response"
    for url in OVERPASS_URLS:
        try:
            print(f" Querying Overpass: {url}")
            r = session.post(url, data={"data": q}, timeout=150)
            if r.status_code != 200:
                last = f"HTTP {r.status_code} from {url}: {r.text[:150]}!"
                print(f" [Overpass fail] {last}")
                continue
            ways = parse_overpass_json(r.json())
            if not ways:
                last = "Overpass returned 0 ways for ref NH 7 / NH 58 in this bbox"
                print(f" [Overpass empty] {last}")
                continue
            Path(ROAD_CACHE_FILE).write_text(
                json.dumps({"bbox": bbox, "fetched": datetime.now(timezone.utc).isoformat(),
                           "query": q, "ways": ways}), encoding="utf-8")
            return ways, f"Overpass API ({len(ways)} ways)"
        except Exception as e: # noqa: BLE001
            last = f"{type(e).__name__}: {e}"
            print(f" [Overpass exception] {url}: {last}")
    print(f" Tip: open https://overpass-turbo.eu, run this query, Export -> GeoJSON, and save it as\n"
          f" {ROAD_GEOJSON_FILE} next to this script:\n" + "\n".join(" " + l for l in q.splitlines()))
    return None, f"OSM unavailable ({last})"

# -----
# NEGATIVE SAMPLING (distance-matched)

```

```

# -----
def sample_negatives(pool_pts, pool_tans, offsets_km, pos_xy, n_target, min_sep_km, rng, ref_tree,
                    min_spacing_km=0.10, passes=8, tol_abs_km=0.05, tol_rel=0.15):
    """
    Take points from the reference-line pool, push each sideways by a distance drawn from the
    positives' own distance-to-line distribution, and keep it only if
    * its MEASURED distance to the reference line (same KD-tree used for the positives) is within
    tolerance of the drawn target (near bends a sideways push can land closer to another segment),
    * it is >= min_sep_km from every positive, and
    * it is >= min_spacing_km from the other negatives.
    """
    pos_tree = cKDTree(pos_xy)
    acc = np.empty((n_target, 2))
    n = 0
    offsets = np.asarray(offsets_km, float)
    offsets = offsets[offsets <= MAX_OFFSET_KM]
    if len(offsets) == 0:
        offsets = np.array([0.0])
    for _ in range(passes):
        for idx in rng.permutation(len(pool_pts)):
            t = pool_tans[idx]
            normal = np.array([-t[1], t[0]])
            off = float(rng.choice(offsets))
            p = pool_pts[idx] + normal * off * rng.choice([-1.0, 1.0])
            if abs(ref_tree.query(p)[0] - off) > max(tol_abs_km, tol_rel * off):
                continue
            if pos_tree.query(p)[0] < min_sep_km:
                continue
            if n and np.min(np.hypot(acc[:n, 0] - p[0], acc[:n, 1] - p[1])) < min_spacing_km:
                continue
            acc[n] = p
            n += 1
            if n >= n_target:
                return acc[:n]
    return acc[:n]

# -----
# TERRAIN (Open-Meteo elevation, finite differences)
# -----
def stencil(lat, lon, h):
    d_lat = h / 110574.0
    d_lon = h / (111320.0 * math.cos(math.radians(lat)))
    return [(lat, lon), (lat + d_lat, lon), (lat - d_lat, lon), (lat, lon + d_lon), (lat, lon - d_lon)]

def cache_key(lat, lon, h):
    return f"{lat:.5f},{lon:.5f},{int(h)}"

def request_elevation(session, coords, max_attempts=5):
    """Return (ok, values, reason). reason == 'bad_request' means: try a smaller batch."""
    params = {"latitude": ", ".join(f"{c[0]:.6f}" for c in coords),
              "longitude": ", ".join(f"{c[1]:.6f}" for c in coords)}
    wait = 2.0
    reason = "unknown"
    for attempt in range(1, max_attempts + 1):
        try:
            r = session.get(OPEN_METEO_ELEV, params=params, timeout=30)
            if r.status_code == 400:
                return False, None, f"bad_request: {r.text[:200]}"
            if r.status_code in (429, 500, 502, 503, 504):
                reason = f"HTTP {r.status_code}"
                retry_after = r.headers.get("Retry-After")
                time.sleep(float(retry_after) if retry_after and retry_after.isdigit() else wait)
                wait = min(wait * 2, 30)
                continue
            r.raise_for_status()
            vals = r.json().get("elevation")
            if isinstance(vals, list) and len(vals) == len(coords) and all(v is not None for v in vals):
                return True, [float(v) for v in vals], "ok"
            reason = f"malformed response: {str(r.text)[:200]}"
            print(f" [Terrain malformed] attempt {attempt}/{max_attempts}: {reason}")
        except Exception as e:
            reason = f"{type(e).__name__}: {e}"
            print(f" [Terrain exception] attempt {attempt}/{max_attempts}: {reason}")
            time.sleep(wait)
            wait = min(wait * 2, 30)
    return False, None, reason

def fetch_terrain(points, session, h, batch_pts=20, pause=0.3):
    """points: list of (lat, lon). Returns (cache_dict, list_of_failed_points)."""
    cache_path = Path(TERRAIN_CACHE_FILE)
    cache = {}
    if cache_path.exists():
        try:
            cache = json.loads(cache_path.read_text(encoding="utf-8"))
        except Exception:
            # noqa: BLE001
            cache = {}
    todo = [p for p in points if cache_key(p[0], p[1], h) not in cache]
    print(f" Terrain: {len(points)} points, {len(points) - len(todo)} cached, {len(todo)} to fetch")
    failed, i, bs = [], 0, max(1, batch_pts)
    while i < len(todo):
        batch = todo[i:i + bs]
        coords = [c for p in batch for c in stencil(p[0], p[1], h)]

```

```

ok, vals, reason = request_elevation(session, coords)
if ok:
    for j, p in enumerate(batch):
        cache[cache_key(p[0], p[1], h)] = vals[j * 5:(j + 1) * 5]
        i += len(batch)
        cache_path.write_text(json.dumps(cache), encoding="utf-8")
        if i % 100 < len(batch):
            print(f"    fetched {i}/{len(todo)}")
            time.sleep(pause)
    elif len(batch) > 1:
        bs = max(1, len(batch) // 2)
        print(f" [Terrain batch failed] {reason} -> retrying with batch of {bs} points")
    else:
        failed.append(batch[0])
        print(f" [Terrain GAVE UP] lat={batch[0][0]:.5f} lon={batch[0][1]:.5f}: {reason}")
        i += 1
return cache, failed

def elevation_check(df):
    """Compare API elevation at the positives with the inventory's own DEM elevation (gross-error detector)."""
    chk = df[(df.label == 1) & df["elevation_m_inventory"].notna()]
    if len(chk) < 5:
        return {}
    diff = (chk["elevation_m"] - chk["elevation_m_inventory"]).abs()
    r = float(np.corrcoef(chk["elevation_m"], chk["elevation_m_inventory"])[0, 1])
    med = float(diff.median())
    # A 90 m DEM vs a 30 m DEM legitimately differs by tens of metres on steep road cuts, so the bar is lenient:
    # this only catches gross errors (swapped lat/lon, wrong units, broken API).
    ok = med <= 150 and r >= 0.90
    return {"n": int(len(chk)), "median_abs_diff_m": med, "pearson_r": r,
            "verdict": "OK" if ok else "CHECK: large mismatch - verify lat/lon order and the elevation API"}

def terrain_features(z5, h):
    zc, zn, zs, ze, zw = z5
    dz_dx = (ze - zw) / (2 * h)
    dz_dy = (zn - zs) / (2 * h)
    grad = math.hypot(dz_dx, dz_dy)
    slope = math.degrees(math.atan(grad))
    if grad == 0:
        a_sin = a_cos = 0.0
    else:
        # bearing (clockwise from north) of the DOWNSLOPE direction (east, north) = (-dz_dx, -dz_dy)
        aspect = math.atan2(-dz_dx, -dz_dy)
        a_sin, a_cos = math.sin(aspect), math.cos(aspect)
    return {
        "elevation_m": zc, "slope_deg": slope, "aspect_sin": a_sin, "aspect_cos": a_cos,
        "local_relief_m": max(z5) - min(z5),
    }

# -----
# MODELS + SPATIAL-BLOCK CV
# -----
def make_lr():
    return make_pipeline(StandardScaler(), LogisticRegression(max_iter=2000, class_weight="balanced"))

def make_rf(seed=42):
    return RandomForestClassifier(n_estimators=400, min_samples_leaf=5, class_weight="balanced_subsample",
                                random_state=seed, n_jobs=-1)

def block_cv(df, cols, model_fn, buffer_km, min_pos_test=3):
    """Leave-one-block-out CV along the road, dropping training points within buffer_km of the test block."""
    oof = np.full(len(df), np.nan)
    per_block = []
    along = df["along_km"].to_numpy()
    y = df["label"].to_numpy()
    X = df[cols].to_numpy(float)
    for b in sorted(df["block_id"].unique()):
        te = (df["block_id"] == b).to_numpy()
        lo, hi = along[te].min(), along[te].max()
        tr = (~te) & ((along < lo - buffer_km) | (along > hi + buffer_km))
        if len(np.unique(y[tr])) < 2 or y[te].sum() < min_pos_test or y[te].sum() == te.sum():
            per_block.append(float("nan"))
            continue
        m = model_fn()
        m.fit(X[tr], y[tr])
        p = m.predict_proba(X[te])[:, 1]
        oof[te] = p
        per_block.append(float(roc_auc_score(y[te], p)))
    ok = ~np.isnan(oof)
    if ok.sum() == 0 or len(np.unique(y[ok])) < 2:
        return {"pooled_auc": float("nan"), "pooled_ap": float("nan"), "top20_capture": float("nan"),
                "block_auc": per_block, "block_auc_mean": float("nan"), "block_auc_std": float("nan"), "n_scored": 0}
    thr = np.quantile(oof[ok], 0.80)
    pos_scored = ok & (y == 1)
    return {
        "pooled_auc": float(roc_auc_score(y[ok], oof[ok])),
        "pooled_ap": float(average_precision_score(y[ok], oof[ok])),
        "top20_capture": float((oof[pos_scored] >= thr).mean()),
        "block_auc": per_block,
        "block_auc_mean": float(np.nanmean(per_block)),
        "block_auc_std": float(np.nanstd(per_block)),
    }

```

```

        "n_scored": int(ok.sum()),
    }

def univariate_auc_ci(x, y, n_boot=500, seed=0):
    """Rank AUC of one feature vs the label, with a 95% bootstrap interval (no model, no CV noise)."""
    rng = np.random.default_rng(seed)
    auc = float(roc_auc_score(y, x))
    idx = np.arange(len(y))
    boots = []
    for _ in range(n_boot):
        pick = rng.choice(idx, len(idx), replace=True)
        if len(np.unique(y[pick])) == 2:
            boots.append(roc_auc_score(y[pick], x[pick]))
    lo, hi = np.percentile(boots, [2.5, 97.5])
    return auc, float(lo), float(hi)

def leak_check(df, cols=("dist_coarse_km", "dist_ref_km")):
    """Distance-to-route alone must not predict the label. Returns (passed, details)."""
    details, passed = {}, True
    y = df["label"].to_numpy()
    for col in dict.fromkeys(cols):
        if col not in df:
            continue
        a, lo, hi = univariate_auc_ci(df[col].abs().to_numpy(), y)
        details[col] = {"auc": a, "ci95": [lo, hi]}
        if abs(a - 0.5) > LEAK_MAX_DEVIATION:
            passed = False
    return passed, details

# -----
# MAIN
# -----
def parse_args(argv=None):
    ap = argparse.ArgumentParser(description=__doc__, formatter_class=argparse.RawDescriptionHelpFormatter)
    ap.add_argument("--inventory", default=INVENTORY_FILE)
    ap.add_argument("--segments", default=SEGMENTS_FILE)
    ap.add_argument("--neg-ratio", type=float, default=3.0, help="negatives per positive (default 3)")
    ap.add_argument("--min-neg-sep-m", type=float, default=250.0, help="min distance from any positive (m)")
    ap.add_argument("--blocks", type=int, default=6, help="spatial blocks along the road for CV")
    ap.add_argument("--buffer-km", type=float, default=2.0, help="gap between train and test blocks")
    ap.add_argument("--limit", type=int, default=None, help="quick test: use only N positives")
    ap.add_argument("--no-osm", action="store_true", help="skip OpenStreetMap, use Mode B directly")
    ap.add_argument("--seed", type=int, default=42)
    ap.add_argument("--batch-pts", type=int, default=20, help="points per elevation request (5 coords each)")
    return ap.parse_args(argv)

def main(argv=None):
    args = parse_args(argv)
    rng = np.random.default_rng(args.seed)
    random.seed(args.seed)
    session = requests.Session()
    session.headers.update({"User-Agent": "nh7-landslide-hackathon/1.0"})

    print("1) Loading inventory and route ...")
    inv_full = load_inventory(args.inventory)
    seg = load_segments(args.segments)
    print(f"    inventory: {len(inv_full)} landslides | route file: {len(seg)} vertices")

    proj = Proj(float(inv_full["latitude"].mean()))
    coarse_xy = proj.xy(seg["latitude"], seg["longitude"])

    inv = inv_full
    n_blocks = args.blocks
    if args.limit:
        inv = inv_full.sample(n=min(args.limit, len(inv_full)), random_state=args.seed).reset_index(drop=True)
        n_blocks = min(n_blocks, 3)
        print(f"    [TEST MODE] using {len(inv)} positives, {n_blocks} blocks")
    pos_xy = proj.xy(inv["latitude"], inv["longitude"])

    d_c, _, along_c, route_len = project_to_polyline(pos_xy, coarse_xy)
    print(f"    route polyline length {route_len:.0f} km (real road is ~247 km; straight chords are shorter)")
    print(f"    positives' distance to your route polyline: median {np.median(d_c):.2f} km, max {d_c.max():.2f} km")
    a_min, a_max = float(along_c.min()), float(along_c.max())

    # ----- road geometry -----
    print("\n2) Road geometry ...")
    bbox = (inv_full.latitude.min() - 0.05, inv_full.longitude.min() - 0.05,
            inv_full.latitude.max() + 0.05, inv_full.longitude.max() + 0.05)
    ways, road_desc = load_road_ways(bbox, session, use_osm=not args.no_osm)
    mode, osm_stats = "B", {}
    ref_pool_pts = ref_pool_tans = None
    ref_tree = None
    if ways:
        pts25, _ = densify_polyline_ways(ways, proj, 0.025)
        tree25 = cKDTree(pts25)
        d_osm, _ = tree25.query(pos_xy)
        share = float((d_osm <= OSM_ACCEPT_DIST_KM).mean())
        osm_stats = {"source": road_desc, "positives_within_300m_share": share,
                    "median_dist_m": float(np.median(d_osm) * 1000), "p90_dist_m": float(np.quantile(d_osm, 0.9) * 1000)}
        print(f"    OSM road ({road_desc}): {share * 100:.0f}% of positives within 300 m "
              f"    f\"median {osm_stats['median_dist_m']:.0f} m\")")
        if share >= OSM_ACCEPT_SHARE:

```

```

mode = "A"
pool_pts, pool_tans = densify_polyline_ways(ways, proj, 0.10)
dpool, _, apool, _ = project_to_polyline(pool_pts, coarse_xy)
keep = (apool >= a_min - 1.0) & (apool <= a_max + 1.0) & (dpool <= 12.0)
ref_pool_pts, ref_pool_tans = pool_pts[keep], pool_tans[keep]
ref_tree = tree25
offsets = d_osm
print(f"    MODE A: negatives sampled on the real road ({len(ref_pool_pts)} candidate points)")
else:
    print("    OSM geometry does not match the inventory well enough -> falling back to Mode B")
else:
    print(f"    {road_desc}")
if mode == "B":
    pool_pts, pool_tans = densify_polyline(coarse_xy, 0.10)
    _, _, apool, _ = project_to_polyline(pool_pts, coarse_xy)
    keep = (apool >= a_min - 1.0) & (apool <= a_max + 1.0)
    ref_pool_pts, ref_pool_tans = pool_pts[keep], pool_tans[keep]
    ref_pts50, _ = densify_polyline(coarse_xy, 0.05)
    ref_tree = cKDTree(ref_pts50)
    offsets, _ = ref_tree.query(pos_xy)
    print("    MODE B: offset-matched sampling around your route polyline. NOTE: negatives are matched "
          "on distance-to-route but are NOT guaranteed to lie on the actual road.")

# ----- negatives -----
print("\n3) Sampling distance-matched negatives ...")
n_target = int(round(args.neg_ratio * len(inv)))
neg_xy = sample_negatives(ref_pool_pts, ref_pool_tans, offsets, pos_xy, n_target,
                          args.min_neg_sep_m / 1000.0, rng, ref_tree)
if len(neg_xy) < n_target:
    print(f"    [WARN] only {len(neg_xy)}/{n_target} negatives could be placed (pool exhausted)")
nlat, nlon = proj.latlon(neg_xy)
print(f"    {len(neg_xy)} negatives (ratio {len(neg_xy) / len(inv):.1f} : 1), "
      f">= {args.min_neg_sep_m:.0f} m from every positive")

pos_df = pd.DataFrame({
    "label": 1, "source": "mey_2024_inventory",
    "latitude": inv["latitude"].to_numpy(), "longitude": inv["longitude"].to_numpy(),
    "landslide_id": inv["landslide_id"].to_numpy() if "landslide_id" in inv else np.arange(len(inv)),
    "elevation_m_inventory": inv["elevation_m_study"].to_numpy() if "elevation_m_study" in inv else np.nan,
})
neg_df = pd.DataFrame({"label": 0, "source": "road_background", "latitude": nlat, "longitude": nlon,
    "landslide_id": np.nan, "elevation_m_inventory": np.nan})
df = pd.concat([pos_df, neg_df], ignore_index=True)
all_xy = proj.xy(df["latitude"], df["longitude"])
d_all, sd_all, along_all, _ = project_to_polyline(all_xy, coarse_xy)
df["dist_coarse_km"], df["along_km"] = d_all, along_all
df["dist_ref_km"] = ref_tree.query(all_xy)[0]

# ----- terrain -----
print("\n4) Terrain from Open-Meteo (90 m DEM) ...")
points = list(zip(df["latitude"].round(5), df["longitude"].round(5)))
cache, failed = fetch_terrain(points, session, SLOPE_OFFSET_M, batch_pts=args.batch_pts)
feats = []
for lat, lon in points:
    z5 = cache.get(cache_key(lat, lon, SLOPE_OFFSET_M))
    feats.append(terrain_features(z5, SLOPE_OFFSET_M) if z5 else {k: np.nan for k in FEATURES})
df = pd.concat([df, pd.DataFrame(feats)], axis=1)
before = len(df)
df = df.dropna(subset=FEATURES).reset_index(drop=True)
print(f"    terrain failures: {len(failed)} | rows dropped: {before - len(df)} | rows kept: {len(df)}")
if df["label"].nunique() < 2 or df["label"].sum() < 10:
    print("ERROR: too few rows survived to continue. Read the failure messages above.")
    return 1

elev_check = elevation_check(df)
if elev_check:
    print(f"    elevation check vs inventory's 30 m DEM: median |diff| {elev_check['median_abs_diff_m']:.0f} m, "
          f"r={elev_check['pearson_r']:.3f} -> {elev_check['verdict']}")

# ----- blocks + CV -----
df["block_id"] = pd.qcut(df["along_km"], q=n_blocks, labels=False, duplicates="drop").astype(int)
print(f"\n5) Spatial-block CV: {df['block_id'].nunique()} blocks along the road, "
      f"{args.buffer_km:.1f} km buffer between train and test")

leak_ok, leak_details = leak_check(df, cols=("dist_ref_km",))
ref_label = "distance to the real road (OSM)" if mode == "A" else "distance to your route polyline"
print(f"    LEAK TEST on {ref_label}: rank AUC of distance alone must be within +-{LEAK_MAX_DEVIATION} of 0.5")
for k, v in leak_details.items():
    print(f"        {k:16s} AUC={v['auc']:.3f} 95% CI [{v['ci95']}[0]:.3f], {v['ci95']}[1]:.3f]")
print(f"    -> {'PASS' if leak_ok else 'WARN: distance still predicts the label; do not trust terrain results'}")
widest = max((v["ci95"][1] - v["ci95"][0]) for v in leak_details.values())
if widest > 0.15:
    print("    (note: confidence interval is wide - too few points for a reliable leak test; use the full run)")
coarse_info = {}
if mode == "A":
    a, lo, hi = univariate_auc_ci(df["dist_coarse_km"].abs().to_numpy(), df["label"].to_numpy())
    coarse_info = {"auc": a, "ci95": [lo, hi]}
    print(f"    (info only) distance to your straight-line polyline: AUC={a:.3f} [{lo:.3f}, {hi:.3f}] - "
          "can reflect real geography (where the true road bends), so it is not an acceptance test in Mode A")

experiments = {
    "slope only (LR)": ([ "slope_deg", make_lr],),
    "elevation only (LR)": ([ "elevation_m", make_lr],),
    "terrain (LR)": (FEATURES, make_lr),
    "terrain (RF)": (FEATURES, lambda: make_rf(args.seed)),
}

```

```

cv = {}
print(f"\n    {'experiment':22s} {'pooled AUC':>10s} {'block AUC mean+-sd':>20s} {'AP':>6s} {'top20% capture':>15s}")
base_ap = float(df["label"].mean())
for name, (cols, fn) in experiments.items():
    r = block_cv(df, cols, fn, args.buffer_km)
    cv[name] = r
    print(f"    {name:22s} {r['pooled_auc']:.10.3f} {r['block_auc_mean']:>10.3f} +- {r['block_auc_std']:.3f} "
          f"    {r['pooled_ap']:.6.3f} {r['top20_capture'] * 100:14.0f}%")
print(f"    (chance: AUC 0.5, AP {base_ap:.2f}, top-20% capture 20%)")

best_name = max(["terrain (LR)", "terrain (RF)", key=lambda k: np.nan_to_num(cv[k]["pooled_auc"], nan=0)])
best_auc = cv[best_name]["pooled_auc"]
skill = "no demonstrated skill" if best_auc < 0.58 else ("weak" if best_auc < 0.70 else "moderate")
print(f"\n    Best terrain model: {best_name}, pooled AUC {best_auc:.3f} -> {skill} (heuristic labels)")

# ----- final model -----
final = make_rf(args.seed) if "RF" in best_name else make_lr()
final.fit(df[FEATURES].to_numpy(float), df["label"].to_numpy())
if "RF" in best_name:
    imp = dict(zip(FEATURES, map(float, final.feature_importances_)))
else:
    coefs = final.named_steps["logisticregression"].coef_[0]
    imp = dict(zip(FEATURES, map(float, coefs)))
print("    feature importance / std. coefficients:", {k: round(v, 3) for k, v in imp.items()})
joblib.dump({"model": final, "feature_columns": FEATURES, "strategy": "static",
            "feature_config": {"terrain_source": "openmeteo", "slope_offset_m": SLOPE_OFFSET_M},
            "model_kind": "RF" if "RF" in best_name else "LR", "feature_importance": imp,
            "model_name": best_name, "created": datetime.now(timezone.utc).isoformat(),
            "trained_on": "Mey et al. 2024 NH-7 inventory (309 slides)"},
            OUT_MODEL)

# ----- outputs -----
df.to_csv(OUT_DATA, index=False, encoding="utf-8")
Path(OUT_CV).write_text(json.dumps({"leak_test": leak_details, "leak_pass": leak_ok, "cv": cv,
                                   "feature_importance": imp, indent=2}, encoding="utf-8"))
make_plots(df, cv, ref_tree, ways is not None and mode == "A", proj, mode)

meta = {
    "strategy": "static", "mode": mode, "feature_columns": FEATURES, "label_column": "label",
    "group_column": "block_id", "spatial_axis_column": "along_km",
    "positives": int(df.label.sum()), "negatives": int((df.label == 0).sum()),
    "neg_ratio_requested": args.neg_ratio, "min_neg_sep_m": args.min_neg_sep_m,
    "n_blocks": int(df.block_id.nunique()), "buffer_km": args.buffer_km,
    "osm": osm_stats, "elevation_check": elev_check,
    "terrain_failures": len(failed), "leak_test_passed": bool(leak_ok),
    "leak_test": leak_details, "coarse_distance_info": coarse_info,
    "best_model": best_name, "best_pooled_auc": best_auc, "skill_heuristic": skill,
    "citation": CITATION,
    "limitations": [
        "Positives are road-blocking landslides, mostly from one rainfall episode (Sept-Oct 2022); ",
        "points, not polygons.",
        "Negatives are background points, not confirmed stable ground.",
        "Terrain from a 90 m DEM with a 150 m finite-difference stencil: coarse for road-cut slopes.",
        "No lithology or road-widening feature, which Mey et al. identify as main controls.",
        "No rainfall feature: apply live rainfall at serving time as an explicit, explainable factor.",
    ],
}
Path(OUT_META).write_text(json.dumps(meta, indent=2), encoding="utf-8")
write_notes(meta, cv, args, len(failed), road_desc)

print("\nSaved:", ", ".join([OUT_DATA, OUT_MODEL, OUT_CV, OUT_META, OUT_NOTES, OUT_PLOTS]))
print("\nNEXT: read DATA_NOTES_static.md, look at diagnostic_plots_static.png, then wire "
      f"{OUT_MODEL} into the backend (static score x live rainfall factor).")
return 0

def densify_polyline_ways(ways, proj, step_km):
    pts, tans = [], []
    for w in ways:
        xy = proj.xy([p[0] for p in w], [p[1] for p in w])
        p, t = densify_polyline(xy, step_km)
        if len(p):
            pts.append(p)
            tans.append(t)
    return np.vstack(pts), np.vstack(tans)

def make_plots(df, cv, ref_tree, have_osm, proj, mode):
    fig, ax = plt.subplots(2, 2, figsize=(13, 9))
    pos, neg = df[df.label == 1], df[df.label == 0]
    if ref_tree is not None:
        ref = ref_tree.data[:, 4]
        la, lo = proj.latlon(ref)
        ax[0, 0].scatter(lo, la, s=1, c="lightgrey", label="road reference" if have_osm else "route polyline")
    ax[0, 0].scatter(neg.longitude, neg.latitude, s=6, c="tab:blue", alpha=0.5, label="negatives")
    ax[0, 0].scatter(pos.longitude, pos.latitude, s=8, c="tab:red", alpha=0.8, label="landslides (Mey 2024)")
    ax[0, 0].set_title(f"Positives vs negatives (Mode {mode})")
    ax[0, 0].legend(markerscale=3, fontsize=8)
    bins = np.linspace(0, max(df.dist_coarse_km.max(), 0.1), 25)
    ax[0, 1].hist(neg.dist_coarse_km.abs(), bins=bins, alpha=0.5, label="negatives", density=True)
    ax[0, 1].hist(pos.dist_coarse_km.abs(), bins=bins, alpha=0.5, label="landslides", density=True)
    ax[0, 1].set_title("Distance to route polyline (must overlap)")
    ax[0, 1].set_xlabel("km")
    ax[0, 1].legend()
    sb = np.linspace(0, max(df.slope_deg.quantile(0.99), 1), 25)
    ax[1, 0].hist(neg.slope_deg, bins=sb, alpha=0.5, label="negatives", density=True)

```

```

ax[1, 0].hist(pos.slope_deg, bins=sb, alpha=0.5, label="landslides", density=True)
ax[1, 0].set_title("Slope (deg) by label")
ax[1, 0].legend()
names = list(cv.keys())
vals = [cv[n]["pooled_auc"] for n in names]
ax[1, 1].barh(names, vals, color="tab:green")
ax[1, 1].axvline(0.5, color="k", ls="--", lw=1)
ax[1, 1].set_xlim(0.3, 1.0)
ax[1, 1].set_title("Spatial-block CV, pooled AUC (0.5 = chance)")
plt.tight_layout()
plt.savefig(OUT_PLOTS, dpi=140)
plt.close(fig)

def write_notes(meta, cv, args, n_failed, road_desc):
    rows = "\n".join(
        f"| {k} | {v['pooled_auc']:.3f} | {v['block_auc_mean']:.3f} +- {v['block_auc_std']:.3f} | "
        f"{v['pooled_ap']:.3f} | {v['top20_capture'] * 100:.0f}% |" for k, v in cv.items())
    leak = "\n".join(f"- {k}: AUC {v['auc']:.3f} (95% CI {v['ci95'][0]:.3f}-{v['ci95'][1]:.3f})"
        for k, v in meta["leak_test"].items())
    ec = meta["elevation_check"]
    elev_line = (f"median |diff| {ec['median_abs_diff_m']:.0f} m, r={ec['pearson_r']:.3f} ({ec['verdict']})"
        if ec else "not available")
    mode_text = ("Negatives lie on the real road (OpenStreetMap geometry), with sideways offsets copied from "
        "the positives' own distance distribution." if meta["mode"] == "A" else
        "OpenStreetMap was unavailable or did not match, so negatives are offset-matched around the "
        "straight-line route polyline. They match positives on distance-to-route but are NOT guaranteed "
        "to lie on the actual road.")
    text = f"### Dataset notes: NH-7 static susceptibility model

## Source and citation
Positives: {CITATION}
Check the supplement's license/terms before redistributing the data, and cite it in the pitch and the paper.

## What this dataset is
- {meta['positives']} positives (surveyed road-blocking landslides) and {meta['negatives']} negatives
  (ratio {meta['negatives'] / meta['positives']:.1f} : 1, at least {args.min_neg_sep_m:.0f} m from any positive).
- Mode {meta['mode']}: {mode_text}
- Road source: {road_desc}
- Features (terrain only): {'', '.join(meta['feature_columns'])}. No rainfall: apply it at serving time.

## Checks run
Leak test (distance-to-route alone must predict nothing; pass if |AUC-0.5| <= {LEAK_MAX_DEVIATION}):
{leak}
Result: {'PASS' if meta['leak_test_passed'] else 'WARN - do not trust the terrain results until this passes'}
Terrain failures: {n_failed}. Elevation vs the inventory's own DEM: {elev_line}

## Spatial-block CV ({meta['n_blocks']} blocks along the road, {meta['buffer_km']} km buffer)
| experiment | pooled AUC | block AUC mean +- sd | AP | top-20% capture |
|---|---|---|---|---|
{rows}
How to read it: pooled AUC ranks points across the whole corridor; block AUC ranks points
within each stretch. Elevation can look strong per block but weak pooled because it rises
steadily towards Joshimath.
Chance: AUC 0.5, top-20% capture 20%. Best model: {meta['best_model']} -> {meta['skill_heuristic']}
(heuristic labels: <0.58 none, <0.70 weak, otherwise moderate).

## Limitations
""" + "\n".join(f"- {l}" for l in meta["limitations"]) + "\n"
Path(OUT_NOTES).write_text(text, encoding="utf-8")

if __name__ == "__main__":
    sys.exit(main())

```

Appendix B: nh7_features.py (324 lines)

Shared feature code: 30 m DEM terrain, rivers/streams, optional Mey layers. Used by steps 2 and 3.

```
"""
Shared feature code for the NH-7 demo model.
Used by add_features.py (training) and score_segments.py (scoring) so both compute
features the exact same way.

Feature groups
  openmeteo : elevation/slope/aspect/relief from the 90 m Open-Meteo API (baseline, no extra installs)
  dem30      : Horn slope, aspect, curvature, relief, TPI from the 30 m Copernicus DEM (needs rasterio)
  streams    : distance to the nearest OSM river / stream (Overpass API)
  litho / faults / widened : optional layers taken from the Mey et al. supplement (needs geopandas)
"""
from __future__ import annotations

import json
import math
import re
from datetime import datetime, timezone
from pathlib import Path

import numpy as np
import pandas as pd
from scipy.spatial import cKDTree

import build_static_dataset as B

DEM_BASE = "https://copernicus-dem-30m.s3.amazonaws.com"
DEM_CACHE_DIR = "dem_cache"
WATERWAYS_CACHE = "nh7_waterways_cache.json"

DEM_FEATURES = ["dem_elev_m", "dem_slope_deg", "dem_slope_max_210m", "dem_aspect_sin", "dem_aspect_cos",
                "dem_curvature", "dem_relief_300m", "dem_tpi_300m"]
NEIGHBOURHOOD_M = 210.0 # Mey et al. (2024) take the hillslope gradient within 210 m of the road
STREAM_FEATURES = ["dist_stream_km", "dist_river_km"]
MIN_LITHO_ROWS = 25 # lithology classes with fewer rows are merged into "other" (all zeros)
UTM44N = "EPSG:32644" # corridor lies in UTM zone 44N (78-84 E)

# -----
# 30 m DEM terrain (pure numpy core)
# -----
class DemTerrain:
    """Terrain attributes from a north-up DEM array (row 0 = north). Pure numpy: easy to test."""

    NAN = {k: np.nan for k in DEM_FEATURES}

    def __init__(self, arr, west, north, res_x_deg, res_y_deg):
        self.arr = np.asarray(arr, dtype="float32")
        self.west, self.north, self.rx, self.ry = float(west), float(north), float(res_x_deg), float(res_y_deg)
        self.h, self.w = self.arr.shape

    def rowcol(self, lat, lon):
        return (int(math.floor((self.north - lat) / self.ry)),
                int(math.floor((lon - self.west) / self.rx)))

    def features(self, lat, lon, half=8):
        row, col = self.rowcol(lat, lon)
        r0, r1, c0, c1 = row - half, row + half + 1, col - half, col + half + 1
        if r0 < 0 or c0 < 0 or r1 > self.h or c1 > self.w:
            return dict(self.NAN)
        win = self.arr[r0:r1, c0:c1].astype(float)
        if not np.isfinite(win).all():
            return dict(self.NAN)
        dx = self.rx * 111320.0 * math.cos(math.radians(lat)) # metres per pixel, east-west
        dy = self.ry * 110574.0 # metres per pixel, north-south

        # Horn gradients for every interior cell of the window (shape 2*half-1 square, centred on the point)
        z = win
        gx = ((z[:-2, 2:] + 2 * z[1:-1, 2:] + z[2:, 2:]) - (z[:-2, :-2] + 2 * z[1:-1, :-2] + z[2:, :-2])) / (8 * dx)
        gy = ((z[:-2, :-2] + 2 * z[1:-1, 1:-1] + z[2:, 2:]) - (z[2:, :-2] + 2 * z[2:, 1:-1] + z[2:, 2:])) / (8 * dy)
        slope_grid = np.degrees(np.arctan(np.hypot(gx, gy)))
        k = half - 1
        yy, xx = np.mgrid[-k:k + 1, -k:k + 1]
        inside = (xx * dx) ** 2 + (yy * dy) ** 2 <= NEIGHBOURHOOD_M ** 2
        slope_max = float(slope_grid[inside].max())

        c = half
        a, b, cc = win[c - 1, c - 1], win[c - 1, c], win[c - 1, c + 1]
        d, e, f = win[c, c - 1], win[c, c], win[c, c + 1]
        g, h, i = win[c + 1, c - 1], win[c + 1, c], win[c + 1, c + 1]
        dz_dx = gx[k, k] # east gradient at the point
        dz_dy = gy[k, k] # north gradient at the point (row 0 is north)
        grad = math.hypot(dz_dx, dz_dy)
        if grad == 0:
            a_sin = a_cos = 0.0
        else:
            bearing = math.atan2(-dz_dx, -dz_dy) # compass bearing of the DOWNSLOPE direction
            a_sin, a_cos = math.sin(bearing), math.cos(bearing)
        near = win[c - 5:c + 6, c - 5:c + 6] # 11x11 pixels ~ 330 m
        return {
            "dem_elev_m": float(e),

```

```

        "dem_slope_deg": math.degrees(math.atan(grad)),
        "dem_slope_max_210m": slope_max, # steepest slope within 210 m
        "dem_aspect_sin": a_sin, "dem_aspect_cos": a_cos,
        "dem_curvature": (d + f - 2 * e) / dx ** 2 + (b + h - 2 * e) / dy ** 2, # >0 = concave hollow
        "dem_relief_300m": float(near.max() - near.min()),
        "dem_tpi_300m": float(e - near.mean()), # <0 = valley/hollow, >0 = ridge/spur
    }

def dem_tile_name(lat_floor: int, lon_floor: int) -> str:
    return f"Copernicus_DSM_COG_10_N{lat_floor:02d}_00_E{lon_floor:03d}_00_DEM"

def download_dem_tile(lat_floor, lon_floor, session, cache_dir=DEM_CACHE_DIR, min_bytes=100_000):
    name = dem_tile_name(lat_floor, lon_floor)
    url = f"{DEM_BASE}/{name}/{name}.tif"
    dest = Path(cache_dir) / f"{name}.tif"
    if dest.exists() and dest.stat().st_size > min_bytes:
        return dest
    dest.parent.mkdir(parents=True, exist_ok=True)
    print(f" Downloading DEM tile {name} (one-time, tens of MB) ...")
    r = session.get(url, stream=True, timeout=300)
    if r.status_code != 200:
        raise RuntimeError(f"HTTP {r.status_code} for {url}")
    tmp = dest.with_suffix(".part")
    with open(tmp, "wb") as fh:
        for chunk in r.iter_content(chunk_size=1 << 20):
            fh.write(chunk)
    if tmp.stat().st_size <= min_bytes:
        raise RuntimeError(f"downloaded file for {name} is suspiciously small ({tmp.stat().st_size} bytes)")
    tmp.replace(dest)
    return dest

def load_dem_mosaic(bbox, session, cache_dir=DEM_CACHE_DIR):
    """
    bbox = (south, west, north, east). Downloads the 1-degree tiles that cover it, mosaics them
    (so points near a tile edge still get full neighbourhoods) and returns a DemTerrain.
    This is the only function that needs rasterio.
    """
    import rasterio
    from rasterio.merge import merge

    s, w, n, e = bbox
    tiles = [download_dem_tile(la, lo, session, cache_dir)
              for la in range(int(math.floor(s)), int(math.floor(n)) + 1)
              for lo in range(int(math.floor(w)), int(math.floor(e)) + 1)]
    datasets = [rasterio.open(p) for p in tiles]
    try:
        mosaic, transform = merge(datasets, bounds=(w, s, e, n))
    finally:
        for ds in datasets:
            ds.close()
    return DemTerrain(mosaic[0], transform.c, transform.f, transform.a, -transform.e)

# -----
# Rivers and streams (OpenStreetMap via Overpass)
# -----
def fetch_waterways(bbox, session):
    """Returns a list of (kind, [(lat, lon), ...]) with kind in {'river', 'stream'}; cached on disk."""
    cache = Path(WATERWAYS_CACHE)
    if cache.exists():
        try:
            data = json.loads(cache.read_text(encoding="utf-8"))
            if data.get("ways"):
                return [(k, [tuple(p) for p in pts]) for k, pts in data["ways"]]
        except Exception as e: # noqa: BLE001
            print(f" [WARN] cannot read {WATERWAYS_CACHE}: {e}")
    s, w, n, e = bbox
    q = ('[out:json][timeout:180];\n'
        f'way["waterway"~"^(river|stream)$"]({s:.4f},{w:.4f},{n:.4f},{e:.4f});\nout geom;')
    last = "no response"
    for url in B.OVERPASS_URLS:
        try:
            print(f" Querying Overpass for rivers/streams: {url}")
            r = session.post(url, data={"data": q}, timeout=240)
            if r.status_code != 200:
                last = f"HTTP {r.status_code}: {r.text[:120]!r}"
                print(f" [Overpass fail] {last}")
                continue
            ways = []
            for el in r.json().get("elements", []):
                geom = el.get("geometry") or []
                kind = (el.get("tags") or {}).get("waterway", "stream")
                if el.get("type") == "way" and len(geom) >= 2:
                    ways.append((kind, [(g["lat"], g["lon"]) for g in geom]))
            if not ways:
                last = "0 waterways returned"
                continue
            cache.write_text(json.dumps({"bbox": bbox, "fetched": datetime.now(timezone.utc).isoformat(),
                                         "ways": ways}), encoding="utf-8")
            print(f" {len(ways)} waterways")
            return ways
        except Exception as e2: # noqa: BLE001
            last = f"{type(e2).__name__}: {e2}"

```

```

        print(f" [Overpass exception] {url}: {last}")
        raise RuntimeError(f"waterways unavailable ({last})")

class WaterwayDistance:
    def __init__(self, ways, proj):
        self.proj = proj
        pts_all, pts_river = [], []
        for kind, pts in ways:
            xy = proj.xy([p[0] for p in pts], [p[1] for p in pts])
            dense, _ = B.densify_polyline(xy, 0.05)
            if len(dense):
                pts_all.append(dense)
                if kind == "river":
                    pts_river.append(dense)
        self.tree_all = cKDTree(np.vstack(pts_all))
        self.tree_river = cKDTree(np.vstack(pts_river)) if pts_river else self.tree_all

    def frame(self, lat, lon):
        xy = self.proj.xy(lat, lon)
        return pd.DataFrame({"dist_stream_km": self.tree_all.query(xy)[0],
                             "dist_river_km": self.tree_river.query(xy)[0]})

# -----
# Optional layers from the Mey et al. supplement (geopandas, lazy import)
# -----
def _safe(name: str) -> str:
    return re.sub(r"^\0-9a-zA-Z+", "_", str(name)).strip("_").lower()[1:40]

def _union(gs):
    return gs.union_all() if hasattr(gs, "union_all") else gs.unary_union

def inspect_layer(path):
    """Print what is inside a vector file so you can choose --litho-field etc."""
    import geopandas as gpd
    g = gpd.read_file(path)
    print(f"\n{path}: {len(g)} features | geometry {sorted(set(g.geometry.geom_type))} | CRS {g.crs}")
    print("columns:", [c for c in g.columns if c != "geometry"])
    print(g.drop(columns="geometry").head(5).to_string())

def layer_features(df, cfg):
    """cfg may contain litho {path, field, levels}, faults {path}, widened {path, query}. Returns new columns."""
    import geopandas as gpd

    out = pd.DataFrame(index=df.index)
    pts = gpd.GeoDataFrame(df[["latitude", "longitude"]].copy(), crs="EPSG:4326",
                           geometry=gpd.points_from_xy(df["longitude"], df["latitude"]))
    pts_utm = pts.to_crs(UTM44N)

    if cfg.get("litho"):
        lc = cfg["litho"]
        polys = gpd.read_file(lc["path"]).to_crs("EPSG:4326")
        joined = gpd.sjoin(pts, polys[["field", "geometry"]], how="left", predicate="within")
        joined = joined[~joined.index.duplicated(keep="first")].reindex(df.index)
        codes = joined[lc["field"]].astype(object).where(joined[lc["field"]].notna(), "unknown").astype(str)
        if not lc.get("levels"):
            counts = codes.value_counts()
            lc["levels"] = [k for k, v in counts.items() if v >= MIN_LITHO_ROWS and k != "unknown"]
        for lv in lc["levels"]:
            out[f"litho_{_safe(lv)}"] = (codes == lv).astype(int)
        out.attrs["litho_unmatched_share"] = float((codes == "unknown").mean())

    if cfg.get("faults"):
        lines = gpd.read_file(cfg["faults"]["path"]).to_crs(UTM44N)
        out["dist_fault_km"] = pts_utm.distance(_union(lines.geometry)) / 1000.0

    if cfg.get("widened"):
        wc = cfg["widened"]
        feats = gpd.read_file(wc["path"])
        if wc.get("query"):
            feats = feats.query(wc["query"])
        feats = feats.to_crs(UTM44N)
        out["near_widened_100m"] = (pts_utm.distance(_union(feats.geometry)) <= 100).astype(int)
    return out

# -----
# One engine for training AND scoring
# -----
def feature_columns(cfg):
    cols = list(DEM_FEATURES) if cfg.get("terrain_source") == "dem30" else list(B.FEATURES)
    if cfg.get("streams"):
        cols += STREAM_FEATURES
    if cfg.get("litho"):
        cols += [f"litho_{_safe(lv)}" for lv in cfg["litho"].get("levels", [])]
    if cfg.get("faults"):
        cols += ["dist_fault_km"]
    if cfg.get("widened"):
        cols += ["near_widened_100m"]
    return cols

class FeatureEngine:
    def __init__(self, cfg, session, bbox, proj=None):

```

```

self.cfg, self.session, self.bbox = cfg, session, bbox
self.proj = proj or B.Proj((bbox[0] + bbox[2]) / 2)
self._dem = None
self._water = None

def dem(self):
    if self._dem is None:
        self._dem = load_dem_mosaic(self.bbox, self.session)
    return self._dem

def water(self):
    if self._water is None:
        self._water = WaterwayDistance(fetch_waterways(self.bbox, self.session), self.proj)
    return self._water

def frame(self, df):
    """Feature frame aligned with df.index. Rows that cannot be computed contain NaN."""
    lat, lon = df["latitude"].to_numpy(float), df["longitude"].to_numpy(float)
    parts = []
    if self.cfg.get("terrain_source") == "dem30":
        dem = self.dem()
        parts.append(pd.DataFrame([dem.features(a, b) for a, b in zip(lat, lon)], index=df.index)[DEM_FEATURES])
    else:
        h = self.cfg.get("slope_offset_m", B.SLOPE_OFFSET_M)
        points = list(zip(np.round(lat, 5), np.round(lon, 5)))
        cache, _ = B.fetch_terrain(points, self.session, h)
        rows = []
        for p in points:
            z5 = cache.get(B.cache_key(p[0], p[1], h))
            rows.append(B.terrain_features(z5, h) if z5 else {k: np.nan for k in B.FEATURES})
        parts.append(pd.DataFrame(rows, index=df.index)[B.FEATURES])
    if self.cfg.get("streams"):
        wf = self.water().frame(lat, lon)
        wf.index = df.index
        parts.append(wf)
    if any(self.cfg.get(k) for k in ("litho", "faults", "widened")):
        parts.append(layer_features(df, self.cfg))
    out = pd.concat(parts, axis=1)
    return out.reindex(columns=feature_columns(self.cfg))

```

Appendix C: add_features.py (243 lines)

Step 2: adds 30 m terrain, rivers and optional Mey layers, compares feature sets, saves the v2 model.

```
#!/usr/bin/env python3
"""
add_features.py - STEP 2: better terrain + extra layers, honest comparison, v2 model
=====
Run after build_static_dataset.py (needs nh7_static_training_data.csv).

python add_features.py --check-dem          # 1-minute smoke test: downloads one DEM tile, prints elevations
python add_features.py                      # 30 m DEM terrain + rivers/streams
python add_features.py --inspect FILE ...    # show columns of the Mey supplement layers (shp/gpkg/geojson)
python add_features.py --litho LITHO.shp --litho-field NAME \
    --faults FAULTS.shp --widened WIDEN.shp --widened-query "widened == 1"

It compares feature sets with the same spatial-block CV as step 1:
  A Open-Meteo 90 m terrain (baseline)      B Copernicus 30 m terrain
  C B + distance to rivers/streams          D C + Mey layers (only if you pass them)
and keeps the SIMPLEST set that beats the previous one by more than 0.01 pooled AUC.

Needs: pip install rasterio      (and geopandas only if you use --litho/--faults/--widened)
"""
from __future__ import annotations

import argparse
import copy
import json
import sys
from datetime import datetime, timezone
from pathlib import Path

import joblib
import numpy as np
import pandas as pd
import requests

import build_static_dataset as B
import nh7_features as F

IN_CSV = "nh7_static_training_data.csv"
OUT_CSV = "nh7_static_training_data_v2.csv"
OUT_MODEL = "nh7_static_model_v2.joblib"
OUT_META = "pipeline_meta_static_v2.json"
OUT_CV = "cv_report_static_v2.json"
OUT_NOTES = "DATA_NOTES_static_v2.md"
MIN_GAIN = 0.01
CORRIDOR_BBOX = (30.00, 78.20, 30.62, 79.60) # south, west, north, east
SMOKE_POINTS = {"Rishikesh": ((30.0869, 78.2676), (300, 500)), "Joshimath": ((30.5564, 79.5663), (1700, 2100))}

def parse_args(argv=None):
    ap = argparse.ArgumentParser(description=__doc__, formatter_class=argparse.RawDescriptionHelpFormatter)
    ap.add_argument("--input", default=IN_CSV)
    ap.add_argument("--check-dem", action="store_true")
    ap.add_argument("--inspect", nargs="+", metavar="FILE")
    ap.add_argument("--no-streams", action="store_true")
    ap.add_argument("--litho", help="lithology polygons from the Mey supplement")
    ap.add_argument("--litho-field", help="column in --litho holding the lithology name/code")
    ap.add_argument("--faults", help="fault lines")
    ap.add_argument("--widened", help="road sections that were widened (lines or polygons)")
    ap.add_argument("--widened-query", help="pandas query that selects widened rows, e.g. 'widened == 1'")
    ap.add_argument("--buffer-km", type=float, default=2.0)
    ap.add_argument("--seed", type=int, default=42)
    return ap.parse_args(argv)

def check_dem(session):
    print("DEM smoke test ...")
    try:
        dem = F.load_dem_mosaic(CORRIDOR_BBOX, session)
    except ImportError:
        print("ERROR: rasterio is not installed. Run: pip install rasterio")
        return 1
    except Exception as e: # noqa: BLE001
        print(f"ERROR while loading the DEM: {type(e).__name__}: {e}")
        return 1
    ok = True
    for name, ((lat, lon), (lo, hi)) in SMOKE_POINTS.items():
        f = dem.features(lat, lon)
        z = f["dem_elev_m"]
        good = lo <= z <= hi
        ok &= good
        print(f" {name:10s} elevation {z:7.1f} m (expected {lo}-{hi}) slope {f['dem_slope_deg']:.1f} deg "
              f"-> {'OK' if good else 'UNEXPECTED'}")
    print("DEM looks right." if ok else "DEM values look wrong: check the tile download and the lat/lon order.")
    return 0 if ok else 1

def main(argv=None):
    args = parse_args(argv)
    session = requests.Session()
    session.headers.update({"User-Agent": "nh7-landslide-hackathon/1.0"})
    if args.inspect:
```

```

        for p in args.inspect:
            F.inspect_layer(p)
        return 0
    if args.check_dem:
        return check_dem(session)

    print("1) Loading step-1 dataset ...")
    df = pd.read_csv(args.input)
    need = {"latitude", "longitude", "label", "along_km", "block_id"} | set(B.FEATURES)
    if not need <= set(df.columns):
        print(f"ERROR: {args.input} is missing columns {sorted(need - set(df.columns))}. Run build_static_dataset.py first.")
        return 1
    print(f"    {len(df)} rows ({int(df.label.sum())} positives)")

    cfg_full = {"terrain_source": "dem30", "streams": not args.no_streams}
    if args.litho:
        if not args.litho_field:
            print("ERROR: --litho needs --litho-field (run --inspect on the file to see its columns)")
            return 1
        cfg_full["litho"] = {"path": args.litho, "field": args.litho_field}
    if args.faults:
        cfg_full["faults"] = {"path": args.faults}
    if args.widened:
        cfg_full["widened"] = {"path": args.widened, "query": args.widened_query}

    bbox = (df.latitude.min() - 0.03, df.longitude.min() - 0.03, df.latitude.max() + 0.03, df.longitude.max() + 0.03)

    print("\n2) Computing 30 m terrain (Copernicus DEM) ...")
    engine = F.FeatureEngine({"terrain_source": "dem30"}, session, bbox)
    try:
        dem_frame = engine.frame(df)
    except ImportError:
        print("ERROR: rasterio is not installed. Run: pip install rasterio (then try --check-dem)")
        return 1
    except Exception as e: # noqa: BLE001
        print(f"ERROR loading the DEM: {type(e).__name__}: {e}\n Try: python add_features.py --check-dem")
        return 1
    print(f"    rows with full 30 m neighbourhood: {int(dem_frame.notna().all(axis=1).sum())}/{len(df)}")

    inv = df[df.label == 1]
    if "elevation_m_inventory" in df and inv["elevation_m_inventory"].notna().sum() >= 5:
        m_om = (inv["elevation_m"] - inv["elevation_m_inventory"]).abs().median()
        m_dem = (dem_frame.loc[inv.index, "dem_elev_m"] - inv["elevation_m_inventory"]).abs().median()
        print(f"    median |elevation - inventory elevation| at the slides: Open-Meteo 90 m {m_om:.0f} m | "
              f"    Copernicus 30 m {m_dem:.0f} m")
        if m_dem > 100:
            print("    [WARN] 30 m DEM disagrees with the inventory by >100 m: check --check-dem before trusting it.")

    parts = [dem_frame]
    streams_ok = False
    if cfg_full["streams"]:
        print("\n3) Distance to rivers/streams (OpenStreetMap) ...")
        try:
            parts.append(F.FeatureEngine({"terrain_source": "none", "streams": True}, session, bbox)
                          .water().frame(df["latitude"].to_numpy(), df["longitude"].to_numpy()).set_index(df.index))
            streams_ok = True
        except Exception as e: # noqa: BLE001
            print(f"    [WARN] streams skipped: {type(e).__name__}: {e}")
    cfg_full["streams"] = streams_ok

    layers_ok = False
    if any(k in cfg_full for k in ("litho", "faults", "widened")):
        print("\n4) Mey supplement layers ...")
        try:
            lf = F.layer_features(df, cfg_full)
            parts.append(lf)
            layers_ok = True
            if "litho_unmatched_share" in lf.attrs:
                print(f"    lithology: {len(cfg_full['litho']['levels'])} classes kept, "
                      f"    {lf.attrs['litho_unmatched_share'] * 100:.0f}% of points outside the polygons")
        except Exception as e: # noqa: BLE001
            print(f"    [WARN] layers skipped: {type(e).__name__}: {e}")
            for k in ("litho", "faults", "widened"):
                cfg_full.pop(k, None)

    fr = pd.concat(parts, axis=1)
    keep = fr.notna().all(axis=1)
    if (~keep).any():
        print(f"\n    dropping {int((~keep).sum())} rows with missing features")
    data = pd.concat([df, fr], axis=1)[keep].reset_index(drop=True)

    # ---- feature sets ----
    cfg_A = {"terrain_source": "openmeteo", "slope_offset_m": B.SLOPE_OFFSET_M}
    cfg_B = {"terrain_source": "dem30"}
    sets = {"A Open-Meteo 90 m terrain": cfg_A, "B Copernicus 30 m terrain": cfg_B}
    if streams_ok:
        sets["C B + rivers/streams"] = {**cfg_B, "streams": True}
    if layers_ok:
        sets["D C + Mey layers"] = copy.deepcopy({k: v for k, v in cfg_full.items() if k != "terrain_source"} | cfg_B)

    print(f"\n5) Spatial-block CV, {data.block_id.nunique()} blocks, {args.buffer_km} km buffer")
    results, report = [], {}
    print(f"    {'feature set':>30s} {'model':>4s} {'pooled AUC':>10s} {'block mean+-sd':>16s} {'AP':>6s} {'top20%':>7s}")
    for name, cfg in sets.items():
        cols = F.feature_columns(cfg)
        best = None

```

```

for mname, fn in (("LR", B.make_lr), ("RF", lambda: B.make_rf(args.seed))):
    r = B.block_cv(data, cols, fn, args.buffer_km)
    report[f"{name} | {mname}"] = r
    print(f"    {name:30s} {mname:4s} {r['pooled_auc']:10.3f} {r['block_auc_mean']:9.3f}+-{r['block_auc_std']:3f} "
          f"{r['pooled_ap']:6.3f} {r['top20_capture'] * 100:6.0f}%")
    if best is None or np.nan_to_num(r["pooled_auc"]) > np.nan_to_num(best[1]):
        best = (mname, r["pooled_auc"])
    results.append((name, cfg, cols, best[0], best[1]))

chosen = results[0]
for cand in results[1:]:
    if cand[4] > chosen[4] + MIN_GAIN:
        chosen = cand
name, cfg, cols, mname, auc = chosen
skill = "no demonstrated skill" if auc < 0.58 else ("weak" if auc < 0.70 else "moderate")
print(f"\n    Chosen (simplest set within {MIN_GAIN} AUC of the best): {name} with {mname}, "
      f"pooled AUC {auc:.3f} -> {skill}")

model = B.make_rf(args.seed) if mname == "RF" else B.make_lr()
model.fit(data[cols].to_numpy(float), data["label"].to_numpy())
if mname == "RF":
    imp = dict(zip(cols, map(float, model.feature_importances_)))
else:
    imp = dict(zip(cols, map(float, model.named_steps["logisticregression"].coef_[0])))
top = sorted(imp.items(), key=lambda kv: -abs(kv[1]))[:5]
print("    strongest features:", ", ".join(f"{k} ({v:+.2f})" for k, v in top))

joblib.dump({"model": model, "feature_columns": cols, "feature_config": cfg, "strategy": "static",
            "model_name": f"{name.strip()} / {mname}", "created": datetime.now(timezone.utc).isoformat(),
            "trained_on": "Mey et al. 2024 NH-7 inventory (309 slides)",
            "model_kind": mname, "feature_importance": imp}, OUT_MODEL)
data.to_csv(OUT_CSV, index=False, encoding="utf-8")
Path(OUT_CV).write_text(json.dumps(report, indent=2), encoding="utf-8")
meta = {"strategy": "static", "version": 2, "chosen_feature_set": name.strip(), "model": mname,
        "feature_columns": cols, "feature_config": cfg, "pooled_auc": auc, "skill_heuristic": skill,
        "group_column": "block_id", "rows": len(data), "positives": int(data.label.sum()),
        "feature_importance": imp, "citation": B.CITATION}
Path(OUT_META).write_text(json.dumps(meta, indent=2), encoding="utf-8")
Path(OUT_NOTES).write_text(
    f"# Dataset notes v2\n\n"
    f"Positives: {B.CITATION}\n\n"
    f"Chosen feature set: **{name.strip()}** ({len(cols)} features) with {mname}; pooled spatial-block AUC "
    f"{auc:.3f} ({skill}; heuristic labels).\n\n"
    f"Feature sets compared (pooled AUC, best model per set):\n"
    + "\n".join(f"- {r[0].strip()}: {r[4]:.3f} ({r[3]})" for r in results)
    + "\n\nCaveats: Copernicus DEM is a surface model (trees/buildings included). Streams come from OpenStreetMap "
    + "and may be incomplete. Lithology/faults/widening only enter if you supplied the Mey layers.\n",
    encoding="utf-8")
print(f"\nSaved: {OUT_CSV}, {OUT_MODEL}, {OUT_META}, {OUT_CV}, {OUT_NOTES}")
print("NEXT: python score_segments.py")
return 0

if __name__ == "__main__":
    sys.exit(main())

```

Appendix D: score_segments.py (209 lines)

Step 3: scores the real road and aggregates to the 18 backend segments.

```
#!/usr/bin/env python3
"""
score_segments.py - STEP 3: turn the trained model into per-segment terrain scores for the backend
=====
Reads : nh7_static_model_v2.joblib (or nh7_static_model.joblib), nh7_road_osm_cache.json (real road, if step 1
        got it), optional --backend-segments CSV (default: the 18 seeded backend segments).
Writes : segment_static_scores.json / .csv -> loaded by risk_service.py at backend start-up.

Method : sample points every 250 m along the road, compute the SAME features used in training, score each point,
        assign points to the 18 backend segments by position along the route, and take the 90th percentile of the
        point scores per segment (a segment is as risky as its worst stretches, not its average).
        The percentile RANK of that value among the 18 segments is the "terrain percentile" (0 = safest, 1 = riskiest).

Backend segments CSV columns (optional): id,name,sequence_order,start_lat,start_lng,end_lat,end_lng
"""
from __future__ import annotations

import argparse
import json
import sys
from datetime import datetime, timezone
from pathlib import Path

import joblib
import numpy as np
import pandas as pd
import requests
from scipy.stats import rankdata

import build_static_dataset as B
import nh7_features as F

# The 18 seeded backend segments (from the backend plan). `mock_risk` is the placeholder value from the mock data.
DEFAULT_SEGMENTS = [
    ("seg_01", "Rishikesh to Shivpuri", 1, 30.0869, 78.2676, 30.1357, 78.3892, 0.15),
    ("seg_02", "Shivpuri to Byasi", 2, 30.1357, 78.3892, 30.1472, 78.4839, 0.35),
    ("seg_03", "Byasi to Kaudiyala", 3, 30.1472, 78.4839, 30.0765, 78.5028, 0.72),
    ("seg_04", "Kaudiyala to Devprayag", 4, 30.0765, 78.5028, 30.1459, 78.5986, 0.42),
    ("seg_05", "Devprayag to Teen Dhara", 5, 30.1459, 78.5986, 30.2015, 78.6820, 0.78),
    ("seg_06", "Teen Dhara to Kirtinagar", 6, 30.2015, 78.6820, 30.2173, 78.7456, 0.20),
    ("seg_07", "Kirtinagar to Srinagar", 7, 30.2173, 78.7456, 30.2224, 78.7845, 0.18),
    ("seg_08", "Srinagar to Sirobagarh", 8, 30.2224, 78.7845, 30.2390, 78.8540, 0.92),
    ("seg_09", "Sirobagarh to Rudraprayag", 9, 30.2390, 78.8540, 30.2844, 78.9811, 0.88),
    ("seg_10", "Rudraprayag to Gauchar", 10, 30.2844, 78.9811, 30.2872, 79.1557, 0.48),
    ("seg_11", "Gauchar to Karnaprayag", 11, 30.2872, 79.1557, 30.2587, 79.2173, 0.25),
    ("seg_12", "Karnaprayag to Langasu", 12, 30.2587, 79.2173, 30.2980, 79.2550, 0.38),
    ("seg_13", "Langasu to Nandprayag", 13, 30.2980, 79.2550, 30.3308, 79.3242, 0.44),
    ("seg_14", "Nandprayag to Chamoli", 14, 30.3308, 79.3242, 30.4074, 79.3524, 0.68),
    ("seg_15", "Chamoli to Birahi", 15, 30.4074, 79.3524, 30.4350, 79.3900, 0.75),
    ("seg_16", "Birahi to Pipalkoti", 16, 30.4350, 79.3900, 30.4297, 79.4304, 0.50),
    ("seg_17", "Pipalkoti to Helang (Tangani)", 17, 30.4297, 79.4304, 30.5280, 79.5380, 0.94),
    ("seg_18", "Helang to Joshimath", 18, 30.5280, 79.5380, 30.5564, 79.5663, 0.81),
]
SEG_COLS = ["id", "name", "sequence_order", "start_lat", "start_lng", "end_lat", "end_lng", "mock_risk"]
PRETTY = {
    "dem_slope_deg": "slope", "slope_deg": "slope", "dem_elev_m": "elevation", "elevation_m": "elevation",
    "dem_tpi_300m": "ridge/valley position (TPI)", "dem_relief_300m": "local relief", "local_relief_m": "local relief",
    "dem_curvature": "terrain curvature", "dist_river_km": "distance to river", "dist_stream_km": "distance to stream",
    "aspect_sin": "slope aspect (east-west)", "aspect_cos": "slope aspect (north-south)",
    "dem_aspect_sin": "slope aspect (east-west)", "dem_aspect_cos": "slope aspect (north-south)",
    "dist_fault_km": "distance to fault", "near_widened_100m": "recent road widening nearby",
}
FACE_VALIDITY = {"seg_08": "Srinagar-Sirobagarh", "seg_09": "Sirobagarh-Rudraprayag"} # documented chronic slide zone

def pretty(name):
    return "lithology " + name[len("litho-"):] if name.startswith("litho-") else PRETTY.get(name, name)

def load_backend_segments(path):
    if path:
        seg = pd.read_csv(path)
        if "mock_risk" not in seg:
            seg["mock_risk"] = np.nan
        return seg[SEG_COLS].sort_values("sequence_order").reset_index(drop=True)
    return pd.DataFrame(DEFAULT_SEGMENTS, columns=SEG_COLS)

def road_points(proj, coarse_xy, a_min, a_max, step_km):
    """Points along the real road (OSM cache) inside the corridor; falls back to the straight route polyline."""
    cache = Path(B.ROAD_CACHE_FILE)
    if cache.exists():
        ways = [[tuple(p) for p in w] for w in json.loads(cache.read_text(encoding="utf-8")).get("ways", [])]
        if ways:
            pts, _ = B.densify_polyline_ways(ways, proj, step_km)
            d, _, along, _ = B.project_to_polyline(pts, coarse_xy)
            keep = (along >= a_min - 1.0) & (along <= a_max + 1.0) & (d <= 12.0)
            if keep.sum() > 20:
                return pts[keep], "real road (OpenStreetMap cache)"
            pts, _ = B.densify_polyline(coarse_xy, step_km)
```

```

return pts, "STRAIGHT-LINE route polyline (no OSM road found: points may not lie on the actual road)"

def main(argv=None):
    ap = argparse.ArgumentParser(description=__doc__, formatter_class=argparse.RawDescriptionHelpFormatter)
    ap.add_argument("--model")
    ap.add_argument("--backend-segments")
    ap.add_argument("--step-km", type=float, default=0.25)
    ap.add_argument("--out", default="segment_static_scores")
    args = ap.parse_args(argv)

    model_path = args.model or ("nh7_static_model_v2.joblib" if Path("nh7_static_model_v2.joblib").exists()
                                else "nh7_static_model.joblib")
    if not Path(model_path).exists():
        print("ERROR: no trained model found. Run build_static_dataset.py (and add_features.py) first.")
        return 1
    bundle = joblib.load(model_path)
    cfg, cols = bundle["feature_config"], bundle["feature_columns"]
    print(f"Model: {bundle['model_name']} | features: {len(cols)} | terrain source: {cfg.get('terrain_source')}")

    bseg = load_backend_segments(args.backend_segments)
    verts = [(bseg.start_lat[0], bseg.start_lng[0])] + list(zip(bseg.end_lat, bseg.end_lng))
    proj = B.Proj(float(np.mean([v[0] for v in verts])))
    route_xy = proj.xy([v[0] for v in verts], [v[1] for v in verts])
    contiguous = all(np.hypot*(proj.xy([bseg.end_lat[i]], [bseg.end_lng[i]])[0]
                                     - proj.xy([bseg.start_lat[i + 1]], [bseg.start_lng[i + 1]])[0])) < 0.1
                  for i in range(len(bseg) - 1))
    if not contiguous:
        print("ERROR: backend segments are not contiguous (each end must equal the next start).")
        return 1
    _, _, total = B.project_to_polyline(route_xy[1:], route_xy)
    cum = np.concatenate([[0.0], np.cumsum(np.hypot(*np.diff(route_xy, axis=0).T))])

    pts_xy, how = road_points(proj, route_xy, 0.0, total, args.step_km)
    print(f"Scoring points: {len(pts_xy)} every ~{args.step_km * 1000:.0f} m on the {how}")

    # assign points to backend segments by position along the route; any segment left empty is scored on its
    # straight chord instead (flagged), because "no data" must never silently look like "safe"
    _, along0, _ = B.project_to_polyline(pts_xy, route_xy)
    seg_idx = np.clip(np.searchsorted(cum[1:], along0, side="left"), 0, len(bseg) - 1)
    source = np.array(["road"] * len(pts_xy), dtype=object)
    extra_xy, extra_idx = [], []
    for i in range(len(bseg)):
        if not (seg_idx == i).any():
            chord, _ = B.densify_polyline(route_xy[i:i + 2], args.step_km)
            extra_xy.append(chord)
            extra_idx += [i] * len(chord)
            print(f" [WARN] {bseg.id[i]}: no road points found; scoring along the straight chord instead "
                  "(not on the road: treat this segment's score with extra caution)")
    if extra_xy:
        pts_xy = np.vstack([pts_xy] + extra_xy)
        seg_idx = np.concatenate([seg_idx, extra_idx]).astype(int)
        source = np.concatenate([source, np.array(["chord"] * len(extra_idx), dtype=object)])

    lat, lon = proj.latlon(pts_xy)
    pts = pd.DataFrame({"latitude": lat, "longitude": lon})
    bbox = (lat.min() - 0.03, lon.min() - 0.03, lat.max() + 0.03, lon.max() + 0.03)
    session = requests.Session()
    session.headers.update({"User-Agent": "nh7-landslide-hackathon/1.0"})
    feats = F.FeatureEngine(cfg, session, bbox, proj).frame(pts)
    ok = feats.notna().all(axis=1).to_numpy()
    print(f"Feature rows complete: {int(ok.sum())}/{len(pts)}")
    if ok.sum() < 50:
        print("ERROR: too few scoring points have complete features. Read the messages above.")
        return 1
    feats, seg_idx, source = feats[ok].reset_index(drop=True), seg_idx[ok], source[ok]

    p = bundle["model"].predict_proba(feats[cols].to_numpy(float))[:, 1]

    imp = bundle.get("feature_importance", {})
    mu, sd = feats[cols].mean(), feats[cols].std().replace(0, 1)
    rows = []
    for i, s in bseg.iterrows():
        m = seg_idx == i
        if m.sum() == 0:
            rows.append({"s.to_dict": s.to_dict(), "n_points": 0, "static_p90": np.nan, "static_mean": np.nan,
                        "driver": "no usable scoring points", "points_source": "none"})
            print(f" [WARN] {s['id']}: no usable scoring points (features failed); it will be reported as no_data")
            continue
        z = (feats.loc[m, cols].mean() - mu) / sd
        if bundle.get("model_kind") == "LR":
            contrib = {c: imp.get(c, 0.0) * z[c] for c in cols}
            top = max(contrib, key=contrib.get)
            driver = f"'high' if {z[top]} > 0 else 'low' {pretty(top)} (z={z[top]:+.1f})"
        else:
            contrib = {c: abs(imp.get(c, 0.0)) * abs(z[c]) for c in cols}
            top = max(contrib, key=contrib.get)
            driver = f"unusual {pretty(top)} for this road ({'high' if z[top] > 0 else 'low'}, z={z[top]:+.1f})"
        rows.append({"s.to_dict": s.to_dict(), "n_points": int(m.sum()), "static_p90": float(np.quantile(p[m], 0.9)),
                    "static_mean": float(p[m].mean()), "driver": driver,
                    "points_source": "chord" if (source[m] == "chord").all() else "road"})
    out = pd.DataFrame(rows)
    valid = out.static_p90.notna()
    ranks = rankdata(out.loc[valid, "static_p90"].to_numpy())
    out["terrain_percentile"] = np.nan
    out.loc[valid, "terrain_percentile"] = (ranks - 1) / max(len(ranks) - 1, 1)

```

```

out["model_name"] = bundle["model_name"]
out["generated"] = datetime.now(timezone.utc).isoformat()

out.to_csv(args.out + ".csv", index=False, encoding="utf-8")
Path(args.out + ".json").write_text(json.dumps(out.replace({np.nan: None}).to_dict("records"), indent=2),
                                    encoding="utf-8")

print(f"\n{'seg':7s}{'name':34s}{'pts':>5s}{'p90':>7s}{'pctile':>8s} {'src':5s} driver")
for _, r in out.iterrows():
    print(f"{r['id']:7s}{r['name'][:33]:34s}{r['n_points']:5d}{r['static_p90']:7.2f}{r['terrain_percentile']:8.2f} "
          f"{r['points_source']:5s} {r['driver']}")
# face validity only: Sirobagarh is a documented chronic slide zone (NOT proof of accuracy)
rank_desc = rankdata(-out.static_p90.fillna(-1))
for sid, label in FACE_VALIDITY.items():
    print(f"face-validity: {label} ({sid}) ranks #{int(rank_desc[(out.id == sid).to_numpy()][0])} of {len(out)} "
          "(documented chronic slide zone; a sanity check, not a validation)")
print(f"\nSaved {args.out}.json and {args.out}.csv -> copy segment_static_scores.json next to risk_service.py")
return 0

if __name__ == "__main__":
    sys.exit(main())

```

Appendix E: risk_service.py (132 lines)

Step 4: live risk = terrain rank + current rainfall, ready to call from /risk-map.

```
"""
risk_service.py - STEP 4: live risk for the backend (/risk-map)
=====
Copy this file and segment_static_scores.json into your FastAPI project (e.g. app/).

    risk_score = TERRAIN_WEIGHT * terrain_percentile + RAIN_WEIGHT * min(rain_mm / RAIN_REF_MM, 1)

* terrain_percentile : 0..1 rank of the segment's modelled terrain susceptibility among the 18 segments
                      (from the trained model; it is a RANKING, not a probability).
* rain_mm            : rain over yesterday + today + tomorrow at the segment midpoint (Open-Meteo forecast API).
* The weights and RAIN_REF_MM are DEMO HEURISTICS, not calibrated and not learned. Say so if asked.
  Replace them with published Himalayan rainfall thresholds, or with the rainfall coefficient from the Mey et al.
  (2024) supplement, if you want to cite a source.
* If the rainfall service is down the segment keeps its terrain-only score and reports rain_status="unavailable".
* DEMO MODE: pass simulate_rain_mm (e.g. /risk-map?simulate_rain_mm=120) to show what a storm would do. Every segment
  is then labelled rain_status="simulated" so it can never be mistaken for a real reading.
```

Wire it into the existing route (example):

```
from app.risk_service import get_live_risk_map

@router.get("/risk-map")
def risk_map(simulate_rain_mm: float | None = None):    # optional demo switch, see DEMO MODE above
    try:
        segments = get_live_risk_map(simulate_rain_mm=simulate_rain_mm)
    except Exception:
        logger.exception("live risk failed; serving seeded mock data instead")
        return existing_mock_response()                # keep your current implementation as the fallback
    return {"total_segments": len(segments),
            "high_risk_count": sum(s["risk_level"] in ("High", "Very High") for s in segments),
            "segments": segments}

"""
from __future__ import annotations

import json
import logging
import time
from datetime import datetime, timezone
from pathlib import Path

import requests

log = logging.getLogger("backend")

SCORES_FILE = Path(__file__).with_name("segment_static_scores.json")
TERRAIN_WEIGHT = 0.6
RAIN_WEIGHT = 0.4
RAIN_REF_MM = 100.0                # 3-day rainfall at which the rain term saturates (demo heuristic)
LEVEL_CUTS = [(0.75, "Very High"), (0.50, "High"), (0.25, "Moderate"), (0.0, "Low")]
CACHE_TTL_S = 1800                # re-query rainfall at most every 30 minutes
OPEN_METEO = "https://api.open-meteo.com/v1/forecast"

_segments_cache = None
_rain_cache = {"t": 0.0, "data": None}

def level_for(score: float) -> str:
    for cut, name in LEVEL_CUTS:
        if score >= cut:
            return name
    return "Low"

def load_segments(path=None):
    global _segments_cache
    if _segments_cache is None:
        path = Path(path or SCORES_FILE)
        _segments_cache = json.loads(path.read_text(encoding="utf-8"))
        log.info("risk_service: loaded %d scored segments from %s", len(_segments_cache), path)
    return _segments_cache

def fetch_rainfall(segments, session=None, simulate_rain_mm=None):
    """Returns {segment_id: {"rain_mm": float | None, "status": "ok" | "unavailable" | "simulated"}}."""
    if simulate_rain_mm is not None:
        return {s["id"]: {"rain_mm": float(simulate_rain_mm), "status": "simulated"} for s in segments}
    now = time.time()
    if _rain_cache["data"] is not None and now - _rain_cache["t"] < CACHE_TTL_S:
        return _rain_cache["data"]
    session = session or requests.Session()
    mids = [((s["start_lat"] + s["end_lat"]) / 2, (s["start_lng"] + s["end_lng"]) / 2) for s in segments]
    params = {"latitude": ", ".join(f"{m[0]:.4f}" for m in mids), "longitude": ", ".join(f"{m[1]:.4f}" for m in mids),
              "daily": "precipitation_sum", "past_days": 1, "forecast_days": 2, "timezone": "Asia/Kolkata"}
    result = {s["id"]: {"rain_mm": None, "status": "unavailable"} for s in segments}
    try:
        r = session.get(OPEN_METEO, params=params, timeout=20)
        r.raise_for_status()
        payload = r.json()
        items = payload if isinstance(payload, list) else [payload]
        if len(items) != len(segments):
```

```

        raise ValueError(f"expected {len(segments)} locations, got {len(items)}")
    for s, item in zip(segments, items):
        vals = [v for v in (item.get("daily", {}).get("precipitation_sum") or []) if v is not None]
        if vals:
            result[s["id"]] = {"rain_mm": float(sum(vals)), "status": "ok"}
    _rain_cache.update(t=now, data=result)
    log.info("risk_service: rainfall refreshed for %d segments", len(segments))
except Exception as e: # noqa: BLE001
    log.warning("risk_service: rainfall unavailable (%s: %s); serving terrain-only scores", type(e).__name__, e)
return result

def compute_segment(seg, rain):
    terrain = seg.get("terrain_percentile")
    terrain_status = "ok" if terrain is not None else "no_data"
    terrain = 0.5 if terrain is None else float(terrain) # missing data is NEUTRAL, never "safest"
    rain_mm = rain["rain_mm"] if rain["status"] in ("ok", "simulated") else None
    rain_index = min(rain_mm / RAIN_REF_MM, 1.0) if rain_mm is not None else 0.0
    score = min(TERRAIN_WEIGHT * terrain + RAIN_WEIGHT * rain_index, 1.0)
    return {
        "id": seg["id"], "name": seg["name"], "sequence_order": seg["sequence_order"],
        "start_coords": {"lat": seg["start_lat"], "lng": seg["start_lng"]},
        "end_coords": {"lat": seg["end_lat"], "lng": seg["end_lng"]},
        "risk_level": level_for(score), "risk_score": round(score, 2),
        "updated_at": datetime.now(timezone.utc).strftime("%Y-%m-%dT%H:%M:%SZ"),
        # transparency fields: extra keys are safe for existing clients to ignore
        "terrain_percentile": round(terrain, 2),
        "terrain_status": terrain_status,
        "terrain_points_source": seg.get("points_source", "road"),
        "terrain_level": level_for(TERRAIN_WEIGHT * terrain),
        "rain_mm_3d": None if rain_mm is None else round(rain_mm, 1),
        "rain_status": rain["status"],
        "main_driver": seg.get("driver"),
        "method": "terrain model ranking + live-rainfall heuristic (not calibrated)",
    }

def get_live_risk_map(session=None, simulate_rain_mm=None):
    segments = load_segments()
    rain = fetch_rainfall(segments, session, simulate_rain_mm)
    return [compute_segment(s, rain[s["id"]]) for s in segments]

```